Chapter 9

Advanced Coupling Strategies

> All you really need to know for the moment is that the universe is a lot more complicated than you might think, even if you start from a position of thinking it's pretty damn complicated in the first place.
>
> Douglas Adams

The full potential of AMUSE becomes clear when we couple codes to solve complex multiphysics or multiscale problems. In this chapter we discuss coupling issues and strategies, and present examples of coupled solutions. Computing requirements increase drastically when running multiphysics problems. As a result, some of the examples presented in this chapter may be quite challenging for a laptop or workstation. Non-linear coupling strategies lead to new applications and greatly increase the power of the framework. We are only at the brink of discovering the large number of possibilities and the complexities of the coupling strategies it enables. Many of these, however, may be challenging to validate. We discuss some basic and a few more advanced hierarchical coupling strategies.

9.1 Code-coupling Strategies

Often, two or more processes or physical phenomena are coupled in a single problem. Nature seems to have a good grasp on how to do this, but in the computer it is often far less clear how to achieve an accurate or efficient solution.

Many simulations entail combining physical domains with very different length and time scales. We have seen numerous examples in the preceding chapters. In Section 8.4.1, we introduced an interlaced temporal discretization scheme as a way to couple dynamics and stellar evolution. So far, we primarily symmetrized the interaction between two codes, on the general principle that resolving the effect of code A on the time step for code B, and vice versa, should produce more accurate results. That approach naturally led us to a drift–kick–drift scheme equivalent to the second-order predictor–corrector or Verlet method (Verlet, 1967; see Sections 4.2.3 and 9.1.1). We will see below that this approach represents much more than just a convenient symmetric coupling; in particular, it provides a symplectic coupling when applied to symplectic

codes (Section 4.2.3.2). We then generalize it to coupling an arbitrary number of codes, even when the modules involved are not symplectic nor even widely separated in scale.

Typically, when two systems interact, they either have comparable scales but different physics, or the same phyiscs but different scales. For clarity, we start with a multi-scale system, in which we identify one or more macroscopic (parent) systems, and one or more microscopic (child) systems. We prefer the parent-child terminology, as it is less confusing in discussions.

A child system is characterised by smaller length and time scales compared to the parent system. The child system then dictates the time resolution (i.e., time step size) and the smallest resolvable distance in the problem. Child systems are numerically expensive because of their short time scales.

The parent system typically has larger length and longer time scales. The longer time scale generally allows for larger numerical time steps, making the parent systems relatively cheap compared to integrating the children. The cost per step, however, can be many times more expensive because the large extent of the parent and the large number of objects. An example might be a stellar cluster with thousands of stars and a size scale on the order of a parsec. The typical time scale in such a system would be about a megayear. If the cluster is part of the Galaxy, with 100 billion stars, and a multi kpc size, the typical time scale is easily 100 times larger than the child system's.

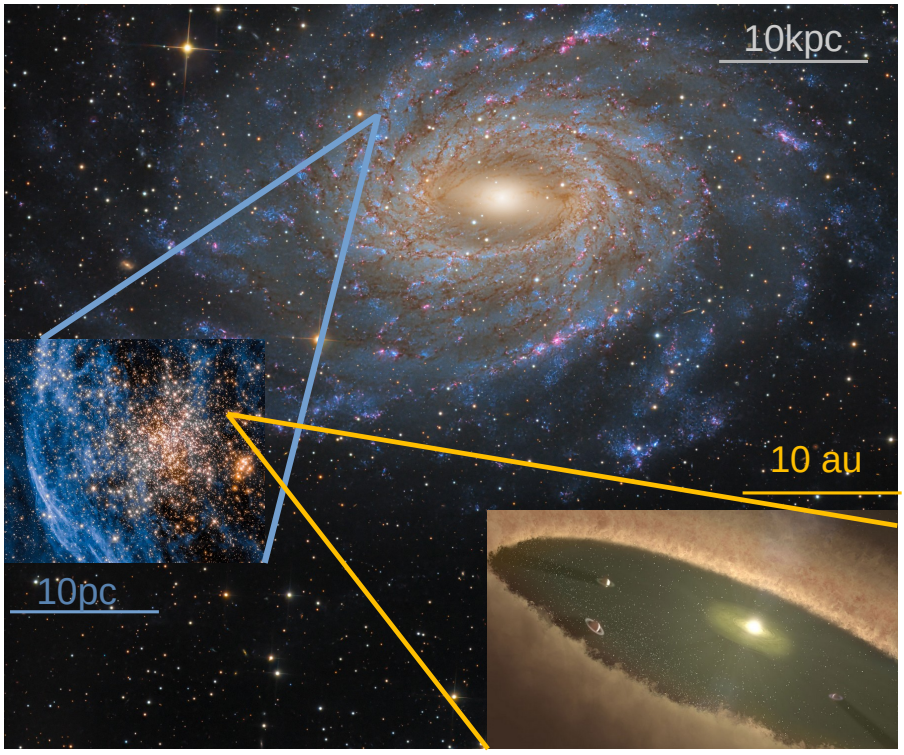

Figure 9.1: Hierarchy of scales illustrated with a hypothetical young planetary system (artist impression of HD 95086 by Spitzer Space Telescope, JPL, NASA, lower right) around a star in a young stellar cluster (illustrated with NGC 1860, NASA, ESA and P. Goudfrooij (STScI); Processing: M. H. Özsaraç (Türkiye Astronomi Dernei), in the galaxy NGC6744 (John Hayes). Images were taken from APOD https://apod.nasa.gov/apod.

In Figure 9.1 we illustrate the hierarchy of scales by presenting an artist's conception of a planetary system, within a young star cluster in a spiral Galaxy. Although the images are real observed systems (the planetary system is an artist's impression, though), they are not actuaally associated. The scales (small bars indicated in the image, represent 10 kpc, 10 pc, and 10 au, for the galaxy, the star cluster and the planetary system, respectively. The associated orbital time scales for these systems will be on the order of 100 Myr, 1 Myr, and 100 yr The Galaxy may contains some 10^{11} stars, the cluster 10^4 stars, and the planetary system several planets.

One scale often dominates in computational expense, memory requirements, communication bandwidth, and storage space. The key to a successful and rewarding multi-scale simulation is to balance these requirements. This scale discrepancy is often addressed by approximating either the parent or the child system, for example using a semi-analytic model. Replacing the child system with an approximate method is often referred to as subgrid physics (whether or not a grid actually exists). Replacing the parent system with an approximate method is often referred to as a background (field) method or external boundary conditions. An example of child subgrid physics is the incorporation of a `multiples` solver (see Section 8.7.1) or a parameterized stellar evolution solver (see Section 5.2) into a gravitational N-body code. An example of a parent background method is approximating the Galactic tidal field by an analytic potential within which a star cluster evolves. Such approximations are inflexible and prevent feedback between child and parent. Our goal in this chapter is to provide a flexible and self-consistent way to realize feedback between the child and parent systems.

In many dynamical cases of interest, the spatial and temporal scales of the child and parent systems are well separated, and we can split the underlying Hamiltonian for the combined system into rapidly and slowly changing parts. Child and parent evolution can then be followed simultaneously by adopting a kick–drift–kick algorithm, allowing both scales to be integrated separately and the results combined into a consistent solution. If we now choose an accurate but expensive method for the child system and a cheap but less accurate method for the parent system, we may gain the ability to save computer time and tune the accuracy, depending on the scale at which we couple the solvers. The resulting *Bridge method* is flexible and dynamic, in the sense that we can control the degree of coupling between solvers, and if necessary, we can introduce a cascade of bridges to address hierarchical coupling, as we will demonstrate in the matrix of coupled codes in `venice` (see Section 10.1.3).

The method can be generalized, both to higher order and to a variety of physical solvers. In principle, the method can have any even order (the odd orders cancel out), but the second-order coupling scheme is sufficiently accurate for most applications.

9.1.1 The Babylonian integrator

In computational astrophysics, probably the most widely used symplectic integrator is the Verlet (leapfrog) method or Babylonian scheme described in Section 4.2.3. Although only of second order, its simplicity and long-term stability make it the method of choice for many applications. It is often implemented in "standard" symplectic form as a kick–drift–kick scheme:

$$\begin{aligned} v' &= v_n + \tfrac{1}{2}a(x_n)\delta t \\ x_{n+1} &= x_n + v'\delta t \\ v_{n+1} &= v' + \tfrac{1}{2}a(x_{n+1})\delta t. \end{aligned} \tag{9.1}$$

A moment's reflection reveals that this is equivalent to the second-order predictor–corrector implementation described earlier. We note that, in practice, the acceleration used in the first kick of the next step is the same as that used in the last kick of the current step (see Section 9.1.1), meaning that only one acceleration need be computed per step, a fact of great importance in large N-body calculations.

An alternative (also symplectic) drift–kick–drift scheme is

$$\begin{aligned} x' &= x_n + \tfrac{1}{2}v_n\delta t \\ v_{n+1} &= v_n + a(x')\delta t \\ x_{n+1} &= x' + \tfrac{1}{2}v_{n+1}\delta t. \end{aligned} \tag{9.2}$$

This approach is much less widely used than its kick–drift–kick cousin, mainly because of its poorer performance in applications where variable time steps are required (Springel, 2005).

Higher-order symplectic schemes also exist (e.g. Yoshida, 1990; Tricarico, 2012; Rein & Spiegel, 2015), and are sometimes employed in astrophysical contexts. The math for these methods goes beyond the scope of this book, but to be complete we explain the higher-order symplectic bridge in Appendix A.4. Finally, we note that time-reversibility in even a non-symplectic scheme is sufficient to ensure that the obtained solution stays close to the true solution and that the error in the energy does not drift. Hut *et al.* (1995) describe an iterative algorithm to make any integrator time-reversible; it is implemented in the AMUSE `smallN` module.

9.1.2 The Bridge Method

In the derivation of the original Bridge scheme, Fujii *et al.* (2007) consider a star cluster (subscript c, indices i and j, total number N_c) orbiting a parent galaxy (subscript g, indices k and l, total number N_g). The internal orbits of stars in the cluster are integrated using a fourth-order Hermite scheme (Makino, 1991) with direct summation of the interstellar gravitational forces. Interactions among the particles comprising the galaxy, and between galaxy particles and cluster stars, are computed using a hierarchical tree force evaluation method (Barnes & Hut, 1986).

To proceed, we need a little dynamics (and in this context, we admit that we exaggerated a little in Section 1.1.3). The Hamiltonian H of the full system can be divided into two parts:

$$H = H_A + H_B, \tag{9.3}$$

where

$$H_A = -\sum_{k<l} \frac{Gm_{gk}m_{gl}}{r_{kl}} - \sum_{k=1}^{N_g}\sum_{i=1}^{N_c} \frac{Gm_{gk}m_{ci}}{r_{ki}} \tag{9.4}$$

is the total gravitational potential energy of the galaxy particles, including their mutual potential energy with the cluster stars, and H_B is the total kinetic energy of the system plus the total internal potential energy of the cluster stars:

$$H_B = \sum_{k=1}^{N_g} \frac{p_{gk}^2}{2m_{gk}} + \sum_{i=1}^{N_c} \frac{p_{ci}^2}{2m_{ci}} - \sum_{i<j} \frac{Gm_{ci}m_{cj}}{r_{ij}}. \tag{9.5}$$

The quantity p is momentum, and r_{ij} is the distance between cluster stars (or galaxy particles) i and j.

Hamilton's equations for a particle can be written as

$$\frac{df}{dt} = D_H f, \tag{9.6}$$

where $f = (\mathbf{x}, \mathbf{p})$ is a six-dimensional phase-space vector representing the particle's dynamical state (position $\mathbf{x}$ and momentum $\mathbf{p} = m\mathbf{v}$), and the differential operator D_H is defined by

$$D_H g \equiv \{g, H\} = \frac{\partial g}{\partial x}\frac{\partial H}{\partial p} - \frac{\partial g}{\partial p}\frac{\partial H}{\partial x} \tag{9.7}$$

for any time-dependent quantity $g(t)$. Here, the syntax $\{.,.\}$ denotes a Poisson bracket (see Landau & Lifshitz, 1969; Section 42). The formal solution to Equation (9.6) is

$$f(t) = e^{tD_H} f(0), \tag{9.8}$$

or, for a single time step,

$$f(t + \delta t) = e^{\delta t D_H}\, f(t), \tag{9.9}$$

This solution is of little use by itself, as we have no explicit way to evaluate it. However, writing $H = H_A + H_B$, and $D_H = D_A + D_B$, it is generally possible to derive an approximation to the exponential:

$$e^{\delta t\,(D_A + D_B)} = \prod_{i=1}^{K} e^{a_i \delta t\, D_A} e^{b_i \delta t\, D_B} + O(\delta t^{K+1}). \tag{9.10}$$

The approximate expression on the right side of Equation (9.10) is symplectic—that is, it preserves phase-space volume and hence is time-reversible (see Section 4.2.3.2 and Section 9.1.1)—and formulae for the coefficients a_i and b_i exist for all orders K, giving rise to a family of practical integration schemes known generically as symplectic integrators.

This all looks like rather scary mathematics, but the applications are really quite simple. For example, the second-order ($K = 2$) version of Equation (9.9) is

$$f(t + \delta t) = e^{\frac{1}{2}\delta t\, D_A}\, e^{\delta t\, D_B}\, e^{\frac{1}{2}\delta t\, D_A} f(t) \tag{9.11}$$

(that is, $a_1 = \frac{1}{2}$, $b_1 = 1$, $a_2 = \frac{1}{2}$, $b_2 = 0$). In a system in which H_A is the total potential energy and H_B is the total kinetic energy of some dynamical system, it is easily shown that $e^{a\tau D_A}$ advances only the velocity by time $a\tau$:

$$\mathbf{v}(t + a\tau) = \mathbf{v}(t) + a\tau\, \mathbf{a}(t) \tag{9.12}$$

(a "kick"), where $\mathbf{a}$ is acceleration, while $e^{b\tau D_B}$ advances only the position:

$$\mathbf{x}(t + b\tau) = \mathbf{x}(t) + b\tau\, \mathbf{v}(t) \tag{9.13}$$

(a "drift"). Thus, the combination of operations in this case is just the kick–drift–kick Verlet–leapfrog scheme discussed in Section 4.2.3.

Applied to the galaxy–cluster system, as discussed in more detail by Fujii *et al.* (2007), this scheme translates to the following algorithm (advancing from time "0" to time "1"):

1. Kick cluster star velocities with time step $\frac{1}{2}\delta t$ using accelerations due to the galaxy; kick galaxy particle velocities with time step $\frac{1}{2}\delta t$ using accelerations due to both galaxy and cluster:

$$\begin{aligned} \mathbf{v}'_c &= \mathbf{v}_{c0} + \tfrac{1}{2}\delta t\, \mathbf{a}_{g\to c0} \\ \mathbf{v}'_g &= \mathbf{v}_{g0} + \tfrac{1}{2}\delta t\, \mathbf{a}_{\mathrm{all}\to g0}. \end{aligned}$$

2. Update (drift) cluster positions and kicked velocities with time step δt, using the internal cluster integration scheme and kicked velocities; update galaxy particle positions with time step δt using kicked velocities:

$$\begin{aligned} \mathbf{x}_{c0} &\to \text{(high-order scheme)} \to \mathbf{x}_{c1} \\ \mathbf{v}'_c &\to \text{(high-order scheme)} \to \mathbf{v}'_{c1} \\ \mathbf{x}_{g1} &= \mathbf{x}_{g0} + \delta t\, \mathbf{v}'_g. \end{aligned}$$

3. Recompute accelerations and complete the step by updating (kicking) all cluster and galaxy velocities:

$$\begin{aligned} \mathbf{v}_{c1} &= \mathbf{v}'_{c1} + \tfrac{1}{2}\delta t\, \mathbf{a}_{g\to c1} \\ \mathbf{v}_{g1} &= \mathbf{v}'_g + \tfrac{1}{2}\delta t\, \mathbf{a}_{\mathrm{all}\to \mathrm{g1}} \end{aligned}$$

This basic scheme carries over to the generalized Bridge class in AMUSE. Note that the overall scheme will be symplectic if and only if the higher-order scheme is symplectic, but even if it is not, the interlaced symmetry of the algorithm still guarantees second-order accurate results.

9.1.3 Implementation of Bridge

The Bridge scheme can be written in Python as presented in the code example below. The main `evolve_model` method has a loop over time in which three tasks are performed; the system is kicked for half a time step, drifted for the full-time step, then kicked again (we saw a similar approach in Section 8.4.2). The source code is shown

below.

```
def evolve_model(time, time_end, time_step):
    first = True
    while self.time < (time_end-time_step/2):
         if first:
             kick(system, partners, time_step/2)
             first = False
         else:
             kick(system, partners, time_step)
         drift(system, time+time_step)
         time = time+time_step
    if not first:
         kick(system, partners, time_step/2)
    return 0
```

The kick operator is composed of two steps. In the first loop, each system is synchronized, then kicked with the function `get_gravity_at_point()` function in the second loop:

```
def kick(systems, partners, time_step):
    for subsystem in systems:
        if do_sync[subsystem]:
            subsystem.synchronize_model()
    for subsystem in systems:
        for y in partners[subsystem]:
            if subsystem is not y:
                kick(subsystem, y.get_gravity_at_point, time_step)
    return 0
```

Code example 9.1 gives an example of `get_gravity_at_point` for a static background potential.

The drift operator evolves each subsystem for the appropriate time step `time_step`:

```
def drift(system, time_end):
    for subsystem in systems:
        time_step = time_end - subsystem.model_time
        subsystem.evolve_model(time_step)
    return 0
```

9.1.4 Determining the bridge time steps

One of the key elements and uncertainties in bridge is the time step. Considerable research went into methods to automatically determine the bridge time step. It is not some parameter easily set and forgotten about; it is important to carefuly choose its proper value.

Over the last several years we have experimented with auto-tuning bridge time steps, but the successes have been marginal. One method entails reinforcement learning, in which a trainable algorithm makes its own decisions on the time step size, considering the computational expense (wall clock time) and energy error produced by the integration (Saz Ulibarrena & Portegies Zwart, 2025).

In Figure 9.2 we present part of the analysis as explored in Saz Ulibarrena & Portegies Zwart (2025), in which a reinforcement algorithm was used to find the optimum bridge time step. the two axes represent integration (wall-clock) time and energy error. The curve shows the trade-off between the two if we reduce the bridge time step.

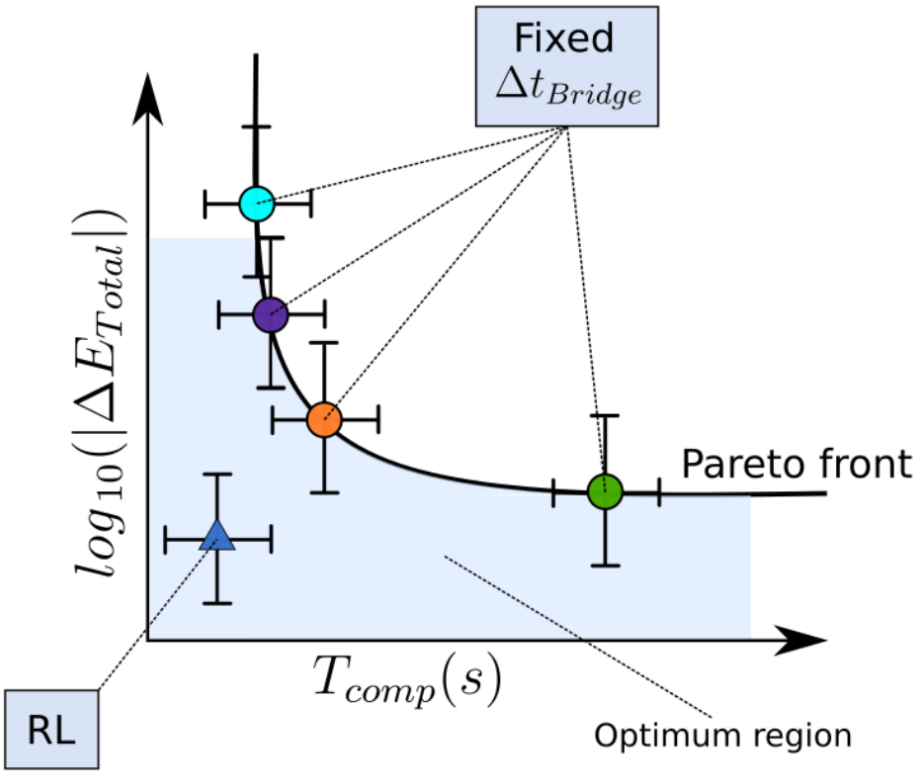

Figure 9.2: Perato curve for the bridge time-step when tuned using reinforcement learning. The bullet points represent fixed bridge time steps, whereas the reinforcement algorithm allows bridge time steps to change with time.

The automated routine to determine the bridge time step, using reinforcement learning, is interesting and in principle powerful. If fully operational, it would have two major advantages: 1) it would not require any expert knowledge to choose a bridge time step, and 2) the calculation would be faster with a smaller overall energy error. In practice, however, it still turns to be hard to find an optimal choice. For now, we are not really encouraging using reinforcement learning to determine your bridge time step, but in the near future we hope that this method can be made into a workable module.

9.2 Using Bridge

Fortunately, you don't have to code all the intricate details of the Bridge method yourself. AMUSE has a `bridge` class that takes care of the details of implementing the classic and higher-order Bridge schemes just described. We illustrate this by developing a simple model of a star cluster moving in an analytic model of the potential near the

Galactic center (McMillan & Portegies Zwart, 2003; Harfst *et al.*, 2010). Then we show how this model can be extended easily to reproduce the seminal results of Fujii *et al.* (2007), which represented the first fully self-consistent treatment of star clusters near the Galactic center.

9.2.1 Star Cluster in a Static Galactic Potential

Code example 9.1 presents a simple class that provides a power-law analytic approximation to the mass distribution and gravitational field within a few hundred parsecs of the Galactic center:

$$\begin{aligned} M(r) &= M_{\rm gal}\left(\frac{r}{R_{\rm gal}}\right)^{\alpha} \\ a(r) &= -\frac{GM(r)}{r^2}, \end{aligned} \tag{9.14}$$

where $M(r)$ is mass interior to radius r and we take $M_{\rm gal} = 1.6\times10^{10}\,{\rm M}_\odot$ within radius $R_{\rm gal} = 1\,{\rm kpc}$, and $\alpha = 1.2$ (Mezger *et al.*, 1996). We initialize the class with

```
galactic_center = GalacticCenterGravityCode(Rgal, Mgal, alpha)
```

```
26 class GalacticCenterGravityCode(object):
27     def __init__(self,R, M, alpha):
28         self.radius=R
29         self.mass=M
30         self.alpha=alpha
31 
32     def get_gravity_at_point(self,eps,x,y,z):
33         r2=x**2+y**2+z**2
34         r=r2**0.5
35         m=self.mass*(r/self.radius)**self.alpha
36         fr=constants.G*m/r2
37         ax=-fr*x/r
38         ay=-fr*y/r
39         az=-fr*z/r
40         return ax,ay,az
41 
42     def circular_velocity(self,r):
43         m=self.mass*(r/self.radius)**self.alpha
44         vc=(constants.G*m/r)**0.5
45         return vc
```

Code example 9.1: Semi-analytic power-law description of the gravitational field in the inner few hundred parsecs of the Galactic center. The quantity `mass` is the mass within distance `radius` of the center, and `alpha` is the slope of the power-law mass distribution.

Key to this (and any) bridge class is the function `get_gravity_at_point()`, which takes a position in space and a softening length as arguments, then returns the gravitational acceleration at the specified location. This function allows any gravity code to be bridged with any other in AMUSE. (In this particular case, the softening length

is ignored, but the argument is required by bridge because it is used by other codes for which softening may be necessary.)

Next, we initialize the star cluster by starting an N-body code and generating a realization of particle masses, positions and velocities. In the example in Code example 9.2, we initialize a code called `nbodycode`, which could be either a direct N-body code, a tree code, or a gravity-enabled SPH code. The cluster is generated from a King (1966) model with `N` particles, total mass `Mcluster`, virial radius `Rcluster`, and a dimensionless potential depth `W0` (see also Section 4.1.3.1). Additional parameters for the N-body code can be passed using the argument `parameters`. In this case, we provide the softening parameter as an argument because we will run this script with a tree code, which requires softening.

```
    Rgal = 1. | units.kpc
    Mgal = 1.6e10 | units.MSun
    alpha = 1.2
    galaxy_code = GalacticCenterGravityCode(Rgal, Mgal, alpha)

    m=galaxy_code.mass*((100|units.pc)/galaxy_code.radius)**galaxy_code.alpha
    cluster_code = make_king_model_cluster(BHTree, N, W0, M, R,
                                           parameters=[("epsilon_squared",
                                                        (0.01 | units.parsec)**2)])

    stars = cluster_code.particles.copy()
    stars.x += Rinit
    stars.vy = 0.8*galaxy_code.circular_velocity(Rinit)
    channel = {"from_framework": stars.new_channel_to(cluster_code.particles),
               "to_framework": cluster_code.particles.new_channel_to(stars)}
    channel["from_framework"].copy_attributes(["x","y","z","vx","vy","vz"])

    plot_cluster(f, stars.x.value_in(units.pc), stars.y.value_in(units.pc), c='r')

    system = bridge(verbose=False)
    system.add_system(cluster_code, (galaxy_code,))

    times = quantities.arange(0|units.Myr, tend, timestep)
    xcom = [] | units.pc
    ycom = [] | units.pc
    for i,t in enumerate(times):
        system.evolve_model(t,timestep=timestep)
        channel["to_framework"].copy_attributes(["x","y","z","vx","vy","vz"])
        com = stars.center_of_mass()
        xcom.append(com[0])
        ycom.append(com[1])

    plt.plot(xcom.value_in(units.pc), ycom.value_in(units.pc), lw=1, c='k')

    x = system.particles.x.value_in(units.parsec)
    y = system.particles.y.value_in(units.parsec)
    cluster_code.stop()
```

Code example 9.2: Initializing the gravity code to be coupled via bridge to a background potential.

The bridge is created by constructing a new `gravity` solver that combines the N-body code `cluster_code` for the star cluster and the analytic `galaxy_code` for the galactic background. The bridge is declared by

```
gravity = bridge.Bridge()
```

and subsequently we identify the codes to be bridged with

```
gravity.add_system(cluster_code, (galaxy_code,))
```

This line adds `cluster_code` as a code to be bridged, and indicates that it interacts with `galaxy_code`. In general, the second argument is a list of codes with which the first code interacts.[1]

In this example, the bridge is unidirectional: the dynamical evolution of the particles in `cluster_code` code is affected by `galaxy_code`, but not the other way around. Multiple codes can be added to a bridge, and their interactions tailored to the problem at hand. This can be very effective if, for example, our galaxy code is composed of several separate parts—for the bulge, bar, and disk, say—and we wanted to model several clusters in the bridge:

```
gravity.add_system(cluster1, (cluster2, cluster3, bulge, bar, disk))
gravity.add_system(cluster2, (cluster1, cluster3, bulge, bar, disk))
gravity.add_system(cluster3, (cluster1, cluster2, bulge, bar, disk))
```

and so on.

Once the bridge is initialized and the integrators added, we set the bridge time step:

```
gravity.timestep = 0.1 | units.Myr
```

It often requires some experimentation to determine what this time step should be. Too long a time step results in inaccurate inclusion of the coupling, whereas too short a time step may make the calculation too expensive. It is possible to change the bridge time step at run time (see Section 10.1.3), but there is currently no ready algorithm to determine its optimum value.

In the event loop over `time`, we now call `evolve_model` from the bridged `gravity` solver

```
gravity.evolve_model(time)
```

Synchronization between the internal code particles and those in the Python script is realized via channels. The event loop is presented in Code example 9.1.

Figure 9.3 shows the results of a simulation of the orbital evolution of the stars in the Arches cluster in the field of the Galactic center. The 50,000 $M_\odot$ star cluster was represented by 1024 equal-mass particles (not a good approximation to the observed mass function of the Arches!) initially represented by a virialized King model (King,

[1] A Python aside: the trailing comma after `galaxy_code` turns the single argument into a tuple that can be evaluated. Forgetting the trailing comma will lead to an error with the inscrutable and somewhat intimidating message `"object is not iterable"`. It is only needed when there is only one item in the list.

```
def make_king_model_cluster(nbodycode, N, W0, Mcluster,
                            Rcluster, parameters = []):

    converter=nbody_system.nbody_to_si(Mcluster,Rcluster)
    bodies=new_king_model(N,W0,convert_nbody=converter)

    code=nbodycode(converter)
    for name,value in parameters:
        setattr(code.parameters, name, value)
    code.particles.add_particles(bodies)
    return code
```

Code example 9.3: Coupling a gravity code to a background potential using a bridge. The last three lines store the stars x and y positions and terminate the code. In the example we adopted the following parameters: `number_of_particles = 1024`, `king_w0 = 3`, `distance_initial = 50.0 | units.pc`, `mass_cluster = 5.0e4 | units.MSun`, and `radius_cluster = 0.8 | units.pc`.

1966) with a radius of 0.8 pc. The cluster had a non-circular orbit in the Galactic field, starting at a distance of 50 pc. We adopted a Barnes–Hut tree code with a rather large softening of 0.1 pc, to allow us to quickly integrate the motion of the stars in the cluster without getting too distracted by the details of the internal cluster dynamics.

9.2.2 Adding extra functionality with a kick to a bridge

Sometimes one would like to add a functionality to the bridge operator in order to kick the system code. A simple example can be considered with the dynamical friction on a black hole of intermediate mass due to its orbit through a galactic nucleus. We discussed a similar issue in Section 4.1.2.3, but without addressing how to add such a term to the integration of an N-body system.

In principle, this process does not work much different than the bridging of two codes, except that in this case one of the codes has no `particle` attribute and doesn't need to be kicked nor drifting itself. In the example, below, we initiate the `Hermite` integrator two intermediate mass black holes that orbit in a background potential. This example is not very different than what we already discussed in Section 9.2.1.

The difference is that we would like to exert a force on the black holes due to the effect of dynamical friction excreted on the black holes. We simply start the N-body code (here `Hermite`), and add the black holes. Then we start the `DynamicalFriction` code, and bridge them together.

```
code1 = Hermite()
code1.particles.add_particles(cluster)
code2 = DynamicalFriction(Coulomb_logarithm=3.7)
system = Bridge(timestep=0.1 | units.Myr)
system.add_system(code1)
system.add_code(code2)
system.evolve_model( 10 | units.Myr )
```

The dynamical friction solver comprises of two parts, a Plummer potential, and a dynamical friction term. We have created a small class to achieve this, in Figure 9.4.

Since the `Hermite` code is 4th order, we use the symbol S_4 (for stars) for the

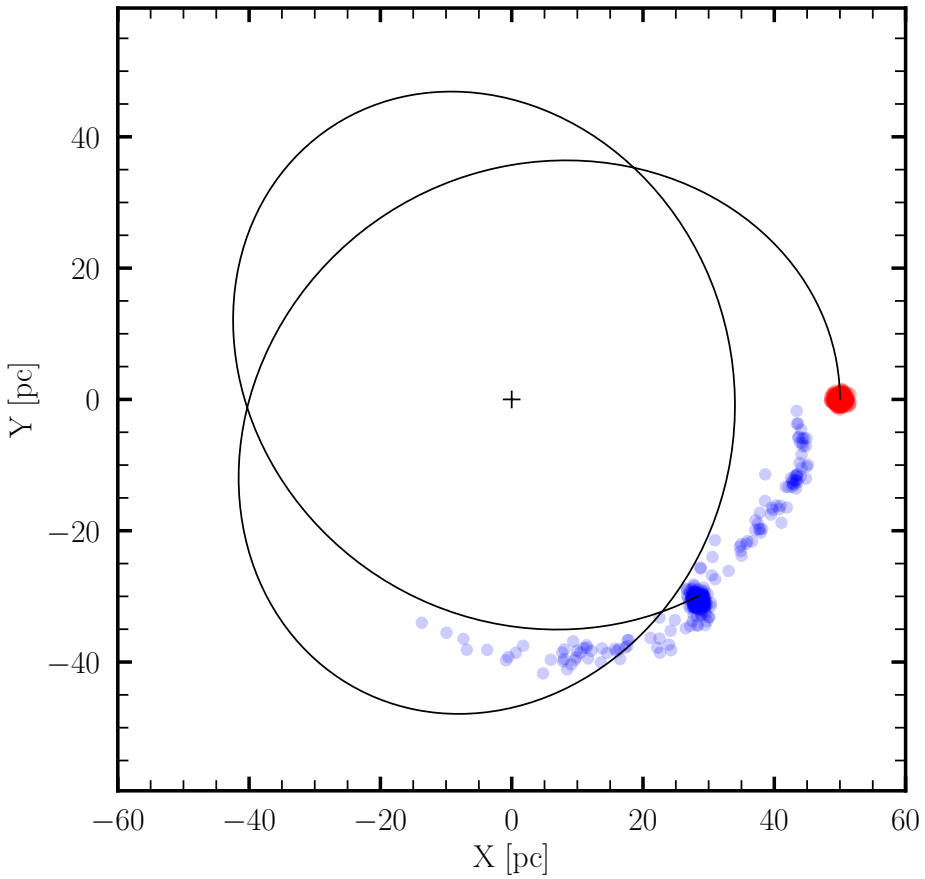

Figure 9.3: Snapshots of an Arches-like cluster orbiting near the Galactic center. Initially (in red) the cluster was composed of 5000 particles taken from a Salpeter mass function over two orders of magnitude with a mean mass of $10\,M_\odot$ leading to a total mass of $50{,}000\,M_\odot$, distributed as a King model in virial equilibrium with $W_0 = 3$ and a virial radius of 0.8 pc. The center of mass of the cluster was placed 50 pc from the Galactic center (along the x-direction) with 80% of the circular velocity (in the y-direction). A softening of 0.1 pc was introduced for all particles in the cluster. The integration was performed using the background galactic potential presented in Code example 9.1. The initialization and main event loop are shown in Code example 9.3 and Code example 9.2, respectively. The equations of motion of the cluster are integrated using the `BHtree` code. The black curve follows the cluster's center of mass, ending in the final shapshot (in blue). The source code for the figure is `amuse.examples.gravity_potential`.

sub-system. The potential is indicated with the letter G (for Galaxy). The bridge is indicated with the left-pointed arrow, and the augmentation is indicated with the letter f (for function) below the bridge arrow. The notation for the augmented bridge in which a 4th order direct N-body code is bridged with a potential including an additional function for (in this case for the dynamical friction term) then has the following notation: $[S_4 \underset{f}{\leftarrow} G]$. this may look somewhat arcane, at first, but once augmented bridge become hierarchical, multi-layers or more complex in other means, this notation captures these complexities nicely.

A star cluster in the Galactic center will feel a frictional force due to the accumulation of stars in its wake. McMillan & Portegies Zwart (2003) used a direct integration technique to study the time scale on which the Arches star cluster sinks from its birth location at about 30 pc from the Galactic center towards the middle of the Milky Way.

It turns out that the time scale depends quite sensitively on the Coulomb parameter $\ln \Lambda$. In their Fig. 6 they present the results of several calculations for the distance to the Galactic center as a function of time for a black hole (or star cluster) of $50\,000\,\mathrm{M}_\odot$. In Figure 9.4 we repeat their calculation but using an external function included in the bridge step

$$[S_4 \underset{f}{\leftarrow} G] \tag{9.15}$$

The equation to solve in $f = f(S, G)$ we calculate the acceleration of the individual particles in the N-body code due to dynamical friction (See Equation 7 of McMillan & Portegies Zwart (2003)):

$$\vec{a}_f = -4\pi \ln(\Lambda) G^2 \rho m \frac{\vec{v}_c}{v_c^3} \chi. \tag{9.16}$$

Here G is the gravitational constant, ρ the local stellar density, with mass m, $\chi \simeq v_c/\sqrt{2}\sigma$ (with σ the local 1-dimensional velocity dispersion), and $\ln(\Lambda) \simeq \ln(r)/\ln(R)$ is the Coulomb logarithm. Here r is the distance of the stars to the potential center (which has size R), and $\vec{v}_c$ is the particle's velocity vector. The acceleration ($\vec{a}_f$) is negative, because dynamical friction results in a drag force leading to deceleration.

The class, performing this calculation together with hovering the stars on the background potential is presented in Figure 9.4.

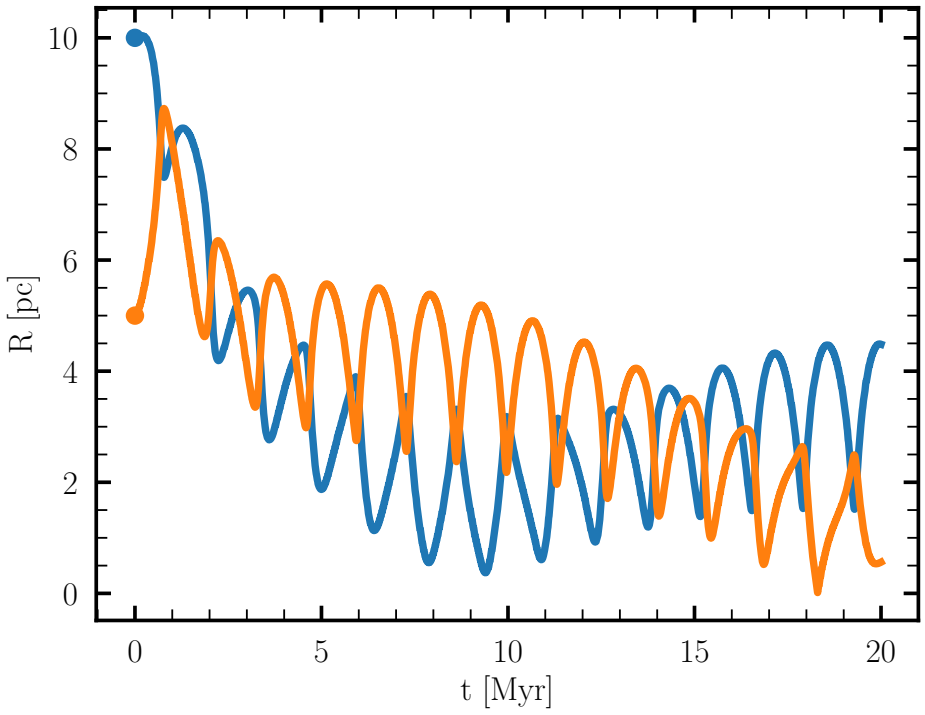

Figure 9.4: Time evolution of Galactocentric distance R of two massive compact objects of $50\,000\,\mathrm{M}_\odot$ each from a distance of 10 pc (blue) and 5 pc (orange). Both curves were integrated using a bridge between `ph4` for integrating the black holes and a background potential with dynamical friction using $\ln(\Lambda) = 3.7$. The sinusoidal variations along the curves are the result of the two black holes not being in a perfect circular orbit. The interference between the two black holes is a consequence of their mutual gravitational interactions. The source code, takes a few seconds to run, can be found at `amuse.examples.imbhs_with_friction`

In this example, we included the dynamical friction as an additional drag force in

the kick of the bridging step. Alternatively, we could include a term in the drift-part of the bridge step. Including additional forces to a bridge can be realized in a similar way by adding an extra term to the kick operator. In this way solving constructing an N-body solver that includes the YORP- and Yarkovski-effects[2] , dynamical friction, tidal evolution, gas-drag, or post-Newtonian dynamics becomes relatively straightforward.

```
13 class CodeWithFriction(bridge.GravityCodeInField):
14
15     def kick_with_field_code(self, particles, field_code, dt):
16         self.LnL = 3.7
17
18         R = particles.position.length()
19         vx = particles.vx.mean()
20         vy = particles.vy.mean()
21         vz = particles.vz.mean()
22         rho = field_code.density(R)
23         vc = field_code.circular_velocity(R)
24         X = 0.34
25         m = particles.mass.sum()
26         ax = -4*np.pi*self.LnL*constants.G**2 * rho*m*(vx/vc**3)*X
27         ay = -4*np.pi*self.LnL*constants.G**2 * rho*m*(vy/vc**3)*X
28         az = -4*np.pi*self.LnL*constants.G**2 * rho*m*(vz/vc**3)*X
29         self.update_velocities(particles, dt, ax, ay, az)
30
31     def drift(self, tend):
32         pass
33
34 def get_system_state_relative_to_SMBH(time, black_holes):
35     t = [] | units.Myr
36     r = [] | units.pc
37     SMBH = black_holes[black_holes.name=="SMBH"]
38     for bi in black_holes-SMBH:
39         t.append(time)
40         r.append((bi.position-SMBH.position).length())
41     return t, r
42
43 def get_system_state(time, black_holes):
44     t = [] | units.Myr
45     r = [] | units.pc
46     for bi in black_holes:
47         t.append(time)
48         r.append(bi.position.length())
49     return t, r
```

Code example 9.4: Example class of a code for adding an extra functionality to a bridge. In this case for calculating the dynamical friction term. Source code is available in `amuse.examples.imbhs_with_friction`.

[2]Named after Ivan Osipovich Yarkovsky (18441902), see also Whipple (1950); Bottke *et al.* (2002).

```
friction_code = CodeWithFriction(
    cluster_gravity,
    (potential,),
    do_sync=True,
    verbose=False,
    radius_is_eps=False,
    h_smooth_is_eps=False,
    zero_smoothing=False,
)
gravity = bridge.Bridge(use_threading=False)
gravity.add_system(cluster_gravity, (potential,) )
gravity.add_code(friction_code)
gravity.timestep = dt_bridge
```

9.2.3 The Classic Bridge

Now that we've seen some of the issues involved in setting up and using a simple Bridge system, we can quite easily create a bridge to reproduce the results of Fujii *et al.* (2007) on the dynamical evolution of a star cluster orbiting the Galactic center. We start by creating a classic Bridge combining a fourth-order Hermite predictor–corrector scheme for the cluster and a Barnes–Hut tree code for the Galaxy. After declaring the codes to be bridged—`ph4` and `bhtree`—we create the bridge solver and add the codes.

```
code1 = Ph4()
code2 = Bhtree()
combined_gravity = bridge.Bridge()
combined_gravity.add_system(code1, (code2,))
combined_gravity.add_system(code2, (code1,))
```

In this case, we have created a bidirectional solver, in which each code interacts with the other.

The simulation is then performed much as before:

```
combined_gravity.timestep = 0.1 | units.Myr
combined_gravity.evolve_model(10 | units.Myr)
```

Figure 9.5 presents an example calculation of the Arches cluster. It is similar to the calculation in Figure 9.3, but now we have adopted a particle representation for the Galactic center region and solved the system using the tree code `BHtree`, in much the same way as presented in Fujii *et al.* (2007). The Arches cluster was initialized with a King model with a dimensionless depth of $W_0 = 3$ and a virial radius of 0.8 pc, as in Figure 9.3. The cluster was placed in a almost circular orbit at a distance of 50 pc from the Galactic center. The latter was represented by a Plummer (1911) sphere with a total mass of 1.6×10^{10} $M_\odot$ and a scale radius of 100 pc (see Section 3.2).

9.2.3.1 Bridging with a Drifter Code

In many cases, it is overkill to run a direct N-body code or tree code in a background potential, particularly if the particles hardly interact, as is the case with planetesimals

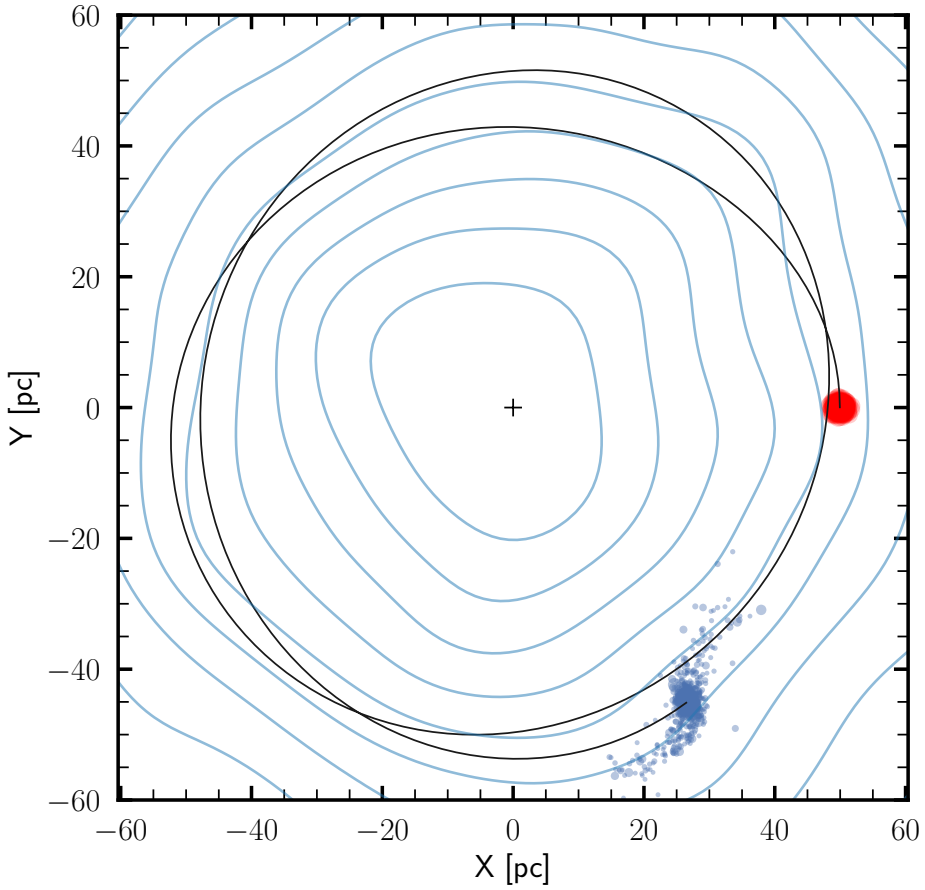

Figure 9.5: Calculation of a star cluster (using `ph4`) in a galaxy (using `BHTree`) for initial conditions comparable to those in Figure 9.3, but now we actively evolve the Galaxy using 10^4 equal mass particles. The star cluster was represented by 1000 point masses drawn from a Salpeter mass function between 0.1 and 100 $M_\odot$. This star cluster, as in Figure 9.3, was born at a distance of 50 pc from the Galactic center (along the X-axis) and with a velocity of 80% of the circular velocity (in the Y-direction). Evolving to an age of 2.5 Myr took about 3 mintes. The source code for the figure is `amuse.examples.gravity_gravity`.

in a planetary system or individual stars in a background galactic potential. In such cases, it is preferable to use a small Python script that just drifts the particles in the background potential, rather than calculating the (negligible) interparticle forces and integrating the orbits. In Code example 9.5 we present an example of such a simple drift code, which we use here to integrate a low-mass star cluster in a semi-analytic Galactic potential. The drift code itself can be used in the same way as any other gravity code. Note that we use the Python `@property` built-in function which allows our small class to remain compatible with past and future versions (see Appendix C.3).

```
class drift_without_gravity:
    def __init__(self, convert_nbody, time=0 | units.Myr):
        self.model_time = time
        self.convert_nbody = convert_nbody
        self.particles = Particles()

    def evolve_model(self, t_end):
        dt = t_end - self.model_time
        self.particles.position += self.particles.velocity * dt
        self.model_time = t_end

    @property
    def potential_energy(self):
        return quantities.zero

    @property
    def kinetic_energy(self):
        return (
            0.5 * self.particles.mass * self.particles.velocity.lengths() ** 2
        ).sum()

    def stop(self):
        pass
```

Code example 9.5: Simple drift code to replace the N-body code in a background potential where interactions between particles are unimportant. Source code is available in `amuse.examples.gravity_drifter`.

The example in Code example 9.5 is far from optimal, and the drifting can easily be done in parallel (for an example, see Appendix A.7.4) or using a graphical processing unit. This would require somewhat more elaborate functions, and you might want to explore the possibility of using

```
from amuse.community.fastkick import Fastkick
```

which is a more efficient and optimized implementation of the kick operation using a GPU. An example of the use of `FastKick` is given in Section 9.3.2.1.

Although the script (which is referenced in Code example 9.5) uses a different model for the background Galactic centrer potential adopted for producing Figure 9.3, the results are also different due to the lack of self-gravity among the stars. As a consequence, the stars stay together longer and the cluster disperses more slowly. We do not recommend using this script for simulating star clusters in a background potential, but it is a handy tool for integrating the orbits of individual (non-self-interacting) particles in the galactic potential.

9.3 Bridging Other Codes

Bridging high-precision gravity with low-precision gravity, or gravity with a background field, is a very effective coupling strategy. Bridge is guaranteed to work in these cases because we can write down a common Hamiltonian for the coupled systems, but Bridge is much more versatile than this. Coupled systems for which we cannot write down a

coupled (or any) Hamiltonian can also be integrated using the classic bridge scheme. We could just as easily have replaced the analytic calculation by another N-body code or a hydrodynamical solver, or bridged one hydrodynamical solver with another, or included radiative transfer. One needs a little courage to do this, because we are unaware of any formal proof that this procedure results in a unique or "correct" solution to the coupled problem. On the other hand, resolution and convergence tests indicate that the solutions obtained in this way do appear to be reliable and accurate, and therefore suitable for scientific interpretation. We leave a complete proof to the brave reader, and should one be forthcoming we will be happy to include it in future editions!

The range of applications of Bridge is up to the imagination of the researcher. In the next section, we present a few bridge topologies you might want to experiment with.

9.3.1 Bridge Hierarchies and Hierarchical Bridges

The simplest possible bridge can be realized by integrating two single particles, such as the Sun and the Earth, and following each—not with a regular integrator, but using Bridge directly. Classic Bridge works as a second-order Verlet–leapfrog integrator, which is perfectly adequate for integrating the solar system, or (almost) any N-body system. An example of how to integrate such a two-body system is given in Code example 9.6.

```
23    star_gravity = Ph4(converter)
24    star_gravity.particles.add_particle(ss[0])
25
26    planet_gravity = Ph4(converter)
27    planet_gravity.particles.add_particle(ss[1:])
28
29    channel_from_star_to_framework = star_gravity.particles.new_channel_to(ss)
30    channel_from_planet_to_framework = planet_gravity.particles.new_channel_to(ss)
31
32    gravity = bridge.Bridge(use_threading=False)
33    gravity.add_system(star_gravity, (planet_gravity,))
34    gravity.add_system(planet_gravity, (star_gravity,))
```

Code example 9.6: Bridge construction for integrating the Sun–Earth system.

In this example, we declare a bridge with two components: one for the Sun, which is affected by Earth's gravity, and one for Earth, which is affected by the gravity of the Sun. The eventual integration is carried out in a loop over time:

```
while time < time_end:
    time += time_step
    gravity.evolve_system(time)
```

in exactly the same way as a more usual integrator or N-body code would be used within the framework. It may seem strange to start an integrator (in this case `ph4`) and have it integrate a single particle (and indeed, some integrators won't even let you do this), but here the integrator simply performs the drift step and defines `get_gravity_at_point()`,

letting the bridge complete the integration of the equations of motion. It probably makes far more sense to use the drifter code in Code example 9.5 instead of the direct N-body code.

We could make this example even stranger by adding the Moon as a third particle and constructing a bridge for all three elements (Sun, Earth, and Moon) as in Code example 9.7, a simple code to integrate a three-body system using second-order bridge. As you might imagine, this is not a very efficient way to integrate the Sun–Earth–Moon system. In fact, for comparable overall accuracy, our three-body bridge is almost 600 times slower than a direct integration using `ph4`, mainly because it is all done in Python, whereas `ph4` is compiled C++ (Stroustrup, 2013).

```
    star_gravity = Ph4(converter)
    star_gravity.particles.add_particle(star)

    planet_gravity = Ph4(converter)
    planet_gravity.particles.add_particle(planet)

    moon_gravity = Ph4(converter)
    moon_gravity.particles.add_particle(moon)

    channel_from_star_to_framework = star_gravity.particles.new_channel_to(ss)
    channel_from_planet_to_framework = planet_gravity.particles.new_channel_to(
        ss)
    channel_from_moon_to_framework = moon_gravity.particles.new_channel_to(ss)

    time = 0 | units.yr
    write_set_to_file(ss, filename,
                      timestamp=time, overwrite_file=True)

    gravity = bridge.Bridge()
    gravity.add_system(star_gravity, (planet_gravity, moon_gravity))
    gravity.add_system(planet_gravity, (star_gravity, moon_gravity))
    gravity.add_system(moon_gravity, (star_gravity, planet_gravity))
```

Code example 9.7: Bridge construction for integrating the Sun–Earth–Moon system. We call this method the multi-bridge; its topology is at the bottom left of Figure 9.6.

We could extend the list of arguments indefinitely, even hierarchically, making the system quite complex. In Code example 9.8, we demonstrate how a hierarchy of level two can be achieved. The bridge for realizing the interaction between Sun and Earth is again bridged with the Moon, leading to a single composite bridge that integrates the entire system. In this case, the `ph4` instantiations take care of the `get_gravity_at_point()` used in the various bridges. This implementation is even slower than the example in Code example 9.6, but no matter—its purpose is to illustrate the method.

The relative performance of the various Bridge implementations just described is presented in Table 9.1. The three bridged methods use `ph4` in various ways, in combination with a bridge. The hierarchical bridge method uses three nested bridges.

Performance may be much better for other bridge topologies or different problems, but the key message here is that Bridge allows considerable flexibility that may com-

```
37      sp_gravity = bridge.Bridge()
38      sp_gravity.add_system(moon_gravity, (planet_gravity,))
39      sp_gravity.add_system(planet_gravity, (moon_gravity,))
40
41      gravity = bridge.Bridge()
42      gravity.add_system(sp_gravity, (star_gravity,))
43      gravity.add_system(star_gravity, (sp_gravity,))
```

Code example 9.8: Hierarchical bridge for integrating the Sun–Earth–Moon system. The majority of the code is identical to Code example 9.7 except that the last four lines there should be replaced with these lines to initialize the hierarchical bridge. This hierarchical-bridge topology is depicted on the bottom right of Figure 9.6.

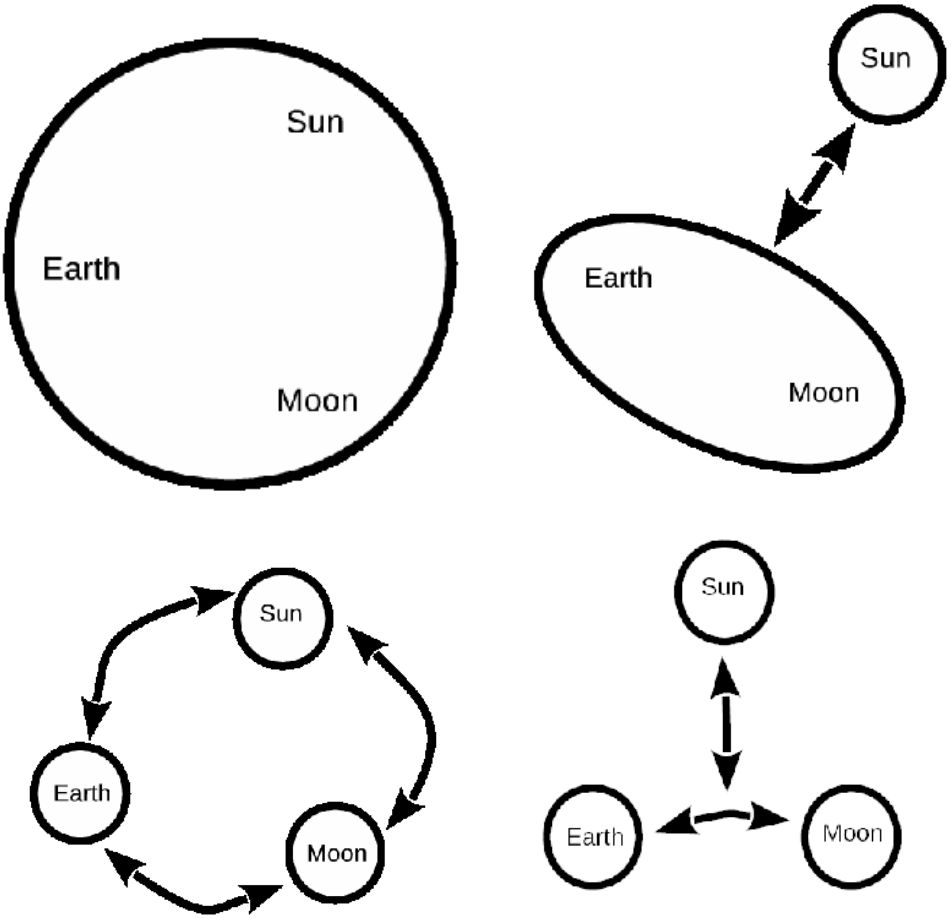

Figure 9.6: The various `Bridge` and `ph4` instantiations in Table 9.1. Top left indicates the use of a single N-body integrator for all three particles. At top right we present a two-way bridge (indicated by the two-pointed arrow) between an N-body realization for only the Sun, and separately for Earth and the Moon. At bottom left we show a three-body bridge, in which each particle is separately handled by an N-body integrator, and each is separately added to a bridge, see Code example 9.7. The bottom right depicts the hierarchical bridge, see Code example 9.8.

pensate for (some) loss of performance in many circumstances. Of course, we don't want performance to drop by a factor of 600, but in practical situations, when running computer-intensive production-quality simulations, the overhead of such complex bridging strategies is actually rather small—generally not more than a few percent.

code	code example	100 yr integration		
		$t_{\rm wall}$ [s]	$\mathrm{d}\Delta\mathrm{E}_{\rm max}$	$\Delta\mathrm{E}_{\rm tot}$
ph4		4.40	4.21×10^{-5}	1.61×10^{-6}
ph4 normalized	4.1	1	1	1
Bridge with ph4	9.6	4.0	0.52	40
Three-body bridge	9.7	8.4	0.57	36
Hierarchical bridge	9.8	12.5	0.57	36
TBB 4th order	A.4	13	1.13	62
TBB 6th order	A.4	4.4	0.52	31
TBB 8th order	A.4	40	0.78	47

Table 9.1: Performance of various Bridge implementations of the Sun–Earth–Moon problem. See also Figure 9.7. Values are relative to `ph4` as a standard, using a `timestep_parameter` of 0.15 (first line), which gives 4.40 s for the wall clock time, $\Delta\mathrm{E}_{\rm max} = 4.21 \times 10^{-5}$ for the energy error, and $\Delta\mathrm{E}_{\rm tot} = -1.61 \times 10^{-6}$ for the maximum energy error per output time step (0.1 yr) and the total energy error for integration over 20 years, respectively. In the following lines we normalize the performance of various bridged integrators (using `ph4`), including higher order bridges. The latter are generally more expensive, but have a favorable energy error per bridge time step. The various scripts are in `amuse.examples` and are called `no_bridge_ph4_integrator`, `three_body_bridge` (in the table named TBB), `three_body_bridge_hierachical`, and `three_body_bridge_n_order`.

Figure 9.7 compares the evolution of the energy error $(E - E_0)/E_0$ of `ph4` with those of the three bridge topologies listed in Table 9.1. To make the comparison more honest, we degraded the `ph4` time step by using a value of 0.6 for `timestep_parameter`. (This parameter is normally set to 0.1, which would have given a relative energy error better than 10^{-12} per step.) It is interesting to note that the energy error in the non-symplectic fourth-order Hermite integrator `ph4` accumulates, whereas the energy errors in the symplectic bridged systems show no secular evolution. The performance of the fourth- and sixth-order bridge methods is quite poor. At present, these are implemented in Python, which is rather inefficient. Porting these to compiled C++ or FORTRAN has not been a priority because, normally, little time is spent in the bridge, except for these somewhat contrived examples.

The example of a bridge coupling or a hierarchical bridge may appear a bit artificial for integrating the Sun–Earth–Moon system, but for other applications, the strict separation of codes, while combining them either sequentially or hierarchically, creates many interesting possibilities. In that sense, the timings listed in Table 9.1 are not representative of the ultimate performance of a more complex simulation in which each of the individual modules is more compute intensive. We encourage the reader to experiment with bridge until bridging is as familiar an operation as eating chicken pie (Cooder, 1972).

If a higher-order bridge is needed, it can be easily implemented by

```
from amuse.couple import bridge
import amuse.ext.composition_methods as CM
gravity = bridge.Bridge(use_threading=False, method=CM.SPLIT_4TH_S_M4)
```

Here, `composition_methods` provides a library for different bridge-coupling strategies (see Appendix A.4.2). The first free parameter in such complex bridge couplings is the number of evaluations of the same bridge step (m). For an example, see Equa-

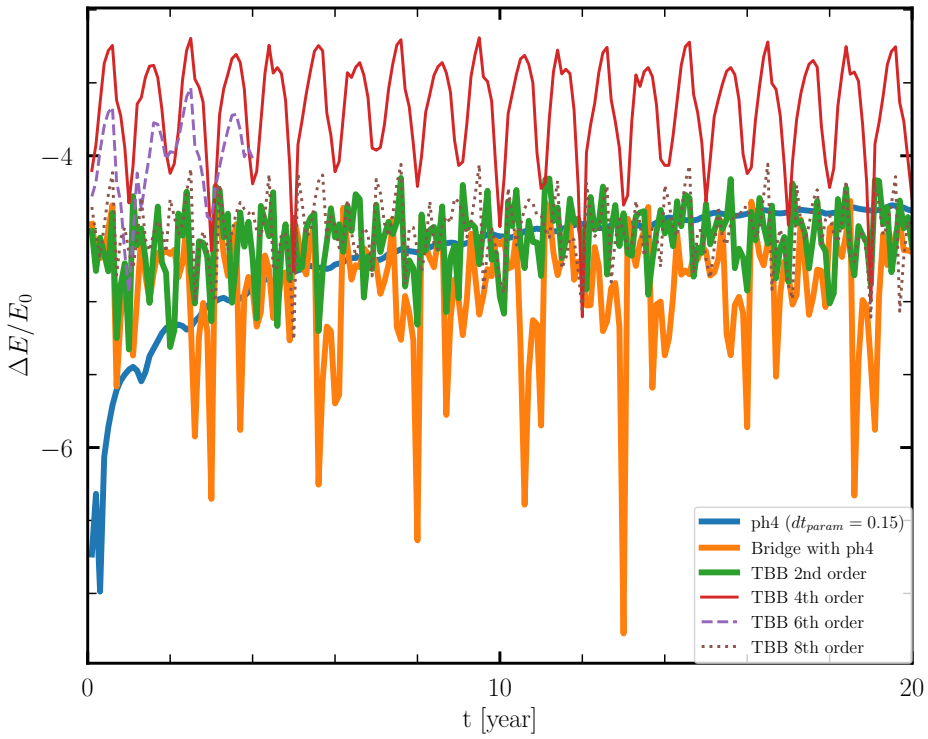

Figure 9.7: Evolution of the energy error while integrating the Sun–Earth–Moon system using a variety of bridge integrators. We use ph4 with a time-step parameter $\eta = 0.15$ (blue curve) as a reference, which clearly shows a secular increase of the energy error in the calculation. The orange curve gives Earth-moon in ph4 but subsequently bridged with the Sun. The other curves are calculated with all particles individually bridged with a 2nd, 4th, 6th and 8th order bridge. Interestingly, the 2nd-order bridge performs best for this case. The source code for reproducing this figure is `amuse.examples.plot_sun_and_earth_and_moon`, but the worker codes are presented in Figure 9.6.

tion (A.10), where we describe a sixth-order bridge with $m = 11$ force evaluations per step. The other parameter is the number of symplectic operations (S, see also Appendix A.4). The argument `method` in the `Bridge` declaration indicates the specific bridge scheme to use, in this case a fourth-order method with $m = 4$; for a sixth-order method one might instead adopt `SPLIT_6TH_SS_M13` (see Table A.16 in Appendix A.4.2).

An example of a higher-order bridge is given in `amuse.examples.three_body_bridge_order4m4`. The default `method` is set to `LEAPFROG`, the classic second-order bridge. Note that, as discussed in Appendix A.4, for codes to be bridgeable to an order higher than second, they must be able to run with negative time steps (stepping backward in time). Not all codes in AMUSE comply with this requirement: those that cannot accommodate negative time steps, such as the Barnes–Hut tree code, can only be bridged to second order.

9.3.2 Bridging Gravity with Hydrodynamics

Many hydrodynamics solvers allow the possibility of adding dark matter particles (called `dm_particles` in AMUSE, as supposed to `gas_particles`; see Section 6.3.3). The `dm_particles` have a gravitational effect on the rest of the system, but no hy-

drodynamical interactions. But the numerical solver used to integrate the equations of motion for `dm_particles` is generally identical to the hydrodynamics solver, which typically is a second-order Verlet integrator with a hierarchical tree to evaluate the forces. These integrators are not very accurate, and they are generally not used for collisional N-body systems. Collisional gravitational problems need more accurate N-body integrators, of which many are available in AMUSE. In principle, if we could couple the high-order gravity modules with the hydrodynamics modules, we could have the best of both worlds: the high accuracy of a fourth-order predictor–corrector Hermite integrator (for example `ph4`), along with the hydrodynamics solver of choice (such as `Fi`). In AMUSE we can do this using Bridge.

Bridging gravity with hydrodynamics is not much different than bridging gravity with gravity, as discussed in Section 9.1. Essentially the same bridge technique can be adopted, and the script will not look much different from one in which two or more gravitational dynamics codes are coupled. We could replace `ph4` in Code example 9.1 with any of the SPH codes in AMUSE without any further changes. Code example 9.9 presents an example of a coupling between an SPH code and a high-order direct N-body code. Here, we generalize the initialization of the hydro and the gravity codes by making them into classes. (For a brief overview of Python classes, we refer to Appendix C.3.7.) This requires a base class, as presented in Code example 9.10, and two specific classes for the gravity and hydrodynamics solvers. These derived classes are presented in Code examples 9.11 and 9.12. Section 9.4.3 presents an example in which we integrate the equations of motion of three stars using an N-body code, while we address the gas dynamics with an SPH code.

```
def gravity_hydro_bridge(Mprim, Msec, a, ecc, t_end, n_steps,
                         Rgas, Mgas, Ngas):

    stars = new_binary_from_orbital_elements(Mprim, Msec, a, ecc,
                                             G=constants.G)
    eps = 1 | units.RSun
    gravity = Gravity(ph4, stars, eps)

    converter = nbody_system.nbody_to_si(1.0|units.MSun, Rgas)
    ism = new_plummer_gas_model(Ngas, convert_nbody=converter)
    ism.move_to_center()
    ism = ism.select(lambda r: r.length()<2*a,["position"])
    hydro = Hydro(Fi, ism, eps)
    model_time = 0 | units.Myr
    filename = "gravhydro.hdf5"
    write_set_to_file(stars.savepoint(model_time), filename, 'amuse')
    write_set_to_file(ism, filename, 'amuse', append_to_file=True)

    gravhydro = bridge.Bridge(use_threading=False)
    gravhydro.add_system(gravity, (hydro,))
    gravhydro.add_system(hydro, (gravity,))
    gravhydro.timestep = 2*hydro.get_timestep()

    while model_time < t_end:
        orbit = orbital_elements_from_binary(stars, G=constants.G)
        dE_gravity = gravity.initial_total_energy/gravity.total_energy
        dE_hydro = hydro.initial_total_energy/hydro.total_energy
        print(("Time:", model_time.in_(units.yr), \
               "ae=", orbit[2].in_(units.AU), orbit[3], \
               "dE=", dE_gravity, dE_hydro))

        model_time += 10*gravhydro.timestep
        gravhydro.evolve_model(model_time)
        gravity.copy_to_framework()
        hydro.copy_to_framework()
        write_set_to_file(stars.savepoint(model_time), filename, 'amuse',
    append_to_file=True)
        write_set_to_file(ism, filename, 'amuse', append_to_file=True)
        print("P=", model_time.in_(units.yr), gravity.particles.x.in_(units.au))
    gravity.stop()
    hydro.stop()
```

Code example 9.9: Combined solver for gravity and hydrodynamics using the generic class structure presented in Code example 9.10 (see `amuse.examples.gravity_hydro` for the full code example).

```
class BaseCode:
    def __init__(self, code, particles, eps=0|units.RSun):

        self.local_particles = particles
        m = self.local_particles.mass.sum()
        l = self.local_particles.position.length()
        self.converter = nbody_system.nbody_to_si(m, l)
        self.code = code(self.converter)
        self.code.parameters.epsilon_squared = eps**2

    def evolve_model(self, time):
        self.code.evolve_model(time)
    def copy_to_framework(self):
        self.channel_to_framework.copy()
    def get_gravity_at_point(self, r, x, y, z):
        return self.code.get_gravity_at_point(r, x, y, z)
    def get_potential_at_point(self, r, x, y, z):
        return self.code.get_potential_at_point(r, x, y, z)
    def get_timestep(self):
        return self.code.parameters.timestep
    @property
    def model_time(self):
        return self.code.model_time
    @property
    def particles(self):
        return self.code.particles
    @property
    def total_energy(self):
        return self.code.kinetic_energy + self.code.potential_energy
    @property
    def stop(self):
        return self.code.stop
```

Code example 9.10: Generic base class for a single code.

```
class Gravity(BaseCode):
    def __init__(self, code, particles, eps=0|units.RSun):
        BaseCode.__init__(self, code, particles, eps)
        self.code.particles.add_particles(self.local_particles)
        self.channel_to_framework \
            = self.code.particles.new_channel_to(self.local_particles)
        self.channel_from_framework \
            = self.local_particles.new_channel_to(self.code.particles)
        self.initial_total_energy = self.total_energy
```

Code example 9.11: Derived class for gravity includes channels.

```
class Hydro(BaseCode):
    def __init__(self, code, particles, eps=0|units.RSun,
                 dt=None, Rbound=None):
        BaseCode.__init__(self, code, particles, eps)
        self.channel_to_framework \
            = self.code.gas_particles.new_channel_to(self.local_particles)
        self.channel_from_framework \
```

```
            = self.local_particles.new_channel_to(self.code.gas_particles)
        self.code.gas_particles.add_particles(particles)
        m = self.local_particles.mass.sum()
        l = self.code.gas_particles.position.length()
        if Rbound is None:
            Rbound = 10*l
        self.code.parameters.periodic_box_size = Rbound
        if dt is None:
            dt = 0.01*numpy.sqrt(l**3/(constants.G*m))
        self.code.parameters.timestep = dt/8.
        self.initial_total_energy = self.total_energy
    @property
    def total_energy(self):
        return self.code.kinetic_energy \
            + self.code.potential_energy \
            + self.code.thermal_energy
```

Code example 9.12: Derived class for hydrodynamics. Hydro codes often require additional parameters, which can be handled in the derived class.

9.3.2.1 Using bridge to relax multi-component systems

In Section 8.5.4.1 we discussed how to use a Henyey code to generate hydrodynamical (SPH) representations of two stars, which we then smashed into one another. After generating hydrodynamical blobs for each star we relaxed both to a glass-like solution to reduce random noise before we started to resolve the collision using an SPH code. Such a relaxation process can be rather elaborate, particularly when a gaseous body is not just self-gravitating but is also affected by the presence of another potential, such as a semi-analytic background or another self-gravitating system of particles.

In Code example 9.13 we demonstrate the use of `bridge` to relax a body of gas in the background potential of a stellar distribution. Bridge is used implicitly here by importing the `relax` routine

```
from amuse.ext.relax_sph import relax
```

which is designed to generate a glass-like solution for a set of SPH particles by evolving them in a background field while imposing critical damping on the particle velocities. The routine `relax` takes several arguments, including the particle set for the gas, the hydrodynamics code, and the code that generates the background potential.

```
79
80
81 def relax_gas_and_stars(stars, gas):
82     dynamical_timescale = gas.dynamical_timescale()
83     converter = nbody_system.nbody_to_si(dynamical_timescale, 1 | units.parsec)
84     hydro = Fi(converter, mode="openmp", redirection="file", redirect_file="fi.log")
85     hydro.parameters.timestep = dynamical_timescale / 100
86     hydro.parameters.eps_is_h_flag = True
87
88     gravity_field_code = Fastkick(converter, mode="cpu", number_of_workers=2)
89     gravity_field_code.parameters.epsilon_squared = (0.01 | units.parsec) ** 2
90     gravity_field_code.particles.add_particles(stars)
91     relaxed_gas = relax(
92         gas,
93         hydro,
94         gravity_field=gravity_field_code,
95         monitor_func=check_energy_conservation,
96         bridge_options=dict(verbose=True, use_threading=False),
97     )
98     gravity_field_code.stop()
99     return hydro
```

Code example 9.13: Using `bridge` to relax a body of gas in the background potential of a stellar distribution. Source code takes a few a minutes to run and is available in `amuse.examples.relax_gas_and_stars`.

The `relax` method uses bridge to couple the hydrodynamics with the background potential, and may require a few tuning parameters. These are specified as `bridge_options`, using a dictionary in the instantiation of `relax`. In Code example 9.13 we choose to increase the amount of diagnostic output from the bridge and to switch off threading. In order to monitor the progress of the relaxation process we also include a user-defined call-back function that will be called after each bridge step. In the current example we keep track of energy conservation while relaxing the gas; the monitoring code is in Code example 9.14. The monitoring function takes at least four arguments: the bridge system (or other code which is used to relax the gas), the current step, the current time, and the total number of steps. The last three arguments are only used for diagnostic output.

```
17 def check_energy_conservation(system, i_step, time, n_steps):
18     unit = units.J
19     U = system.potential_energy.value_in(unit)
20     Q = system.thermal_energy.value_in(unit)
21     K = system.kinetic_energy.value_in(unit)
22     print(
23         f"Step {i_step} of {n_steps}, t={time.as_quantity_in(units.yr)}: "
24         f"U={U:.2e}, Q={Q:.2e}, K={K:.2e} {unit}"
25     )
```

Code example 9.14: Call-back function to monitor energy conservation during the `relax` procerss. Source code is part of available in `amuse.examples.relax_gas_and_stars`.

Code example 9.13 uses another interesting and handy utility in AMUSE, `FastKick`. This routine is actually a rather simple function that computes the kick in the bridge

system, in this case, for the gravity of the stars. Bridge requires such a code, but in this example it would be overkill to use a full N-body code to calculate the kicks between the stars and the gas particles, because we don't need to move the stars and therefore can omit the N^2 operation in the gravity module. Using a direct N-body code, or in some cases even a tree code to resolve the kicks would in fact make the code considerably slower. (You can easily verify this by replacing `FastKick` by any gravity solver from Chapter 4.) `FastKick` is parallel (using the keyword `number_of_workers`), and has a GPU implementation (invoked by setting `mode = "gpu"`:

```
gravity_field_code = FastKick(converter, mode="gpu", number_of_workers=4)
```

The result is a fast and consistent kick implementation.

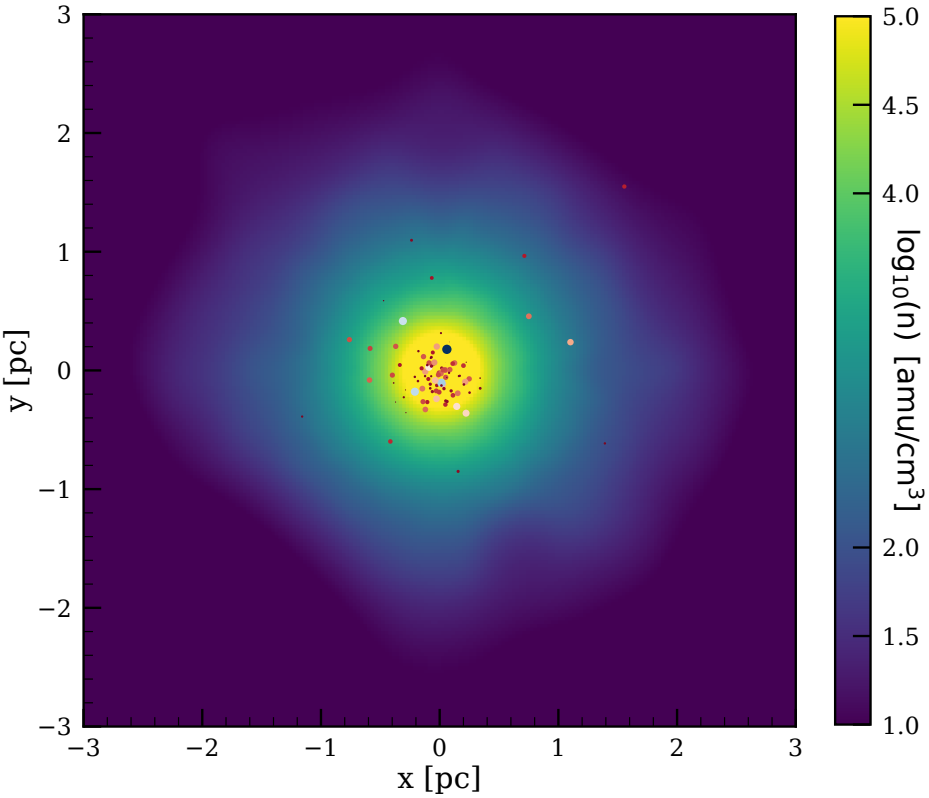

Figure 9.8: Initial conditions for stars and gas in equilibrium. The stars are distributed according to a Plummer (1911) sphere. The gas was initially also generated from a Plummer sphere, but subsequently the gas around each star up to the total mass of that star was removed. The resulting three-dimensional distribution then resembles Tynjetaler cheese. After the gas around the stars was removed we brought the gas distribution into equilibrium with the stars using the `relax` routine in Code example 9.13. The source code to reproduce this figure is `amuse.examples.relax_gas_and_stars`, which takes about 2 hours to run.

Figure 9.8 presents the results of generating initial conditions for 100 stars in a Plummer distribution with a virial radius of 0.33 pc with masses from a Salpeter (1955) function between 1 $M_\odot$ and 100 $M_\odot$. The gas was initially distributed identically to the stars with 10^5 equal mass particles but with a mass ten times higher than the total mass in stars. After generating the gas distribution we iteratively removed the gas particles around each star with a mass equal to the mass of that particular star. These

88941 remaining gas particles were then relaxed. This method was used by Pelupessy & Portegies Zwart (2012) to generate the initial conditions for their simulations. In Figure 6.10 we presented the results of a hydrodynamical calculation in which we allowed the gas to clump and form stars through sink particles. The two distributions are quite different, and it is not clear what gives the most satisfactory initial conditions for further simulations.

Generating initial conditions in this way can be expensive in terms of computer time, but that is sometimes the price to pay for a realistic or at least a desirable realization.

9.4 Examples

9.4.1 Dissolving star cluster in the Galactic potential

The Sun was probably born in a star cluster with a mass of 500–10,000 $M_\odot$ and a virial radius of about 1 pc (Portegies Zwart, 2009). In the next few subsections we explore the orbital evolution of the Sun and its parental cluster over the past 4.6 Gyr. This work is motivated by the idea that some of the Sun's siblings may still be around in the solar neighborhood and may be identifiable (Portegies Zwart, 2009). As has been pointed out by Mishurov & Acharova (2011), this may not be so easy due to the lumpiness of the Galaxy. In addition, the Galaxy may have changed considerably since the Sun's birth and reconstruction of the comoving Galactic potential over the last 4.6 Gyr is not trivial. Nevertheless, let's explore this interesting question for ourselves.

We perform a series of simulations of a small cluster with 1000 stars orbiting in the Galactic disk, for a variety of representations of the Galactic potential. The basic approach is to choose a starting point 4.6 Gyr ago and then follow the orbits of the Sun and its siblings forward in time, to assess their current locations with respect to the Sun.

9.4.1.1 Dissolution of a star cluster in a smooth Galaxy potential

We first adopt a three-component model of the Galactic potential from Bovy (2015), consisting of a bulge, modeled as a power-law density profile with an exponential cut-off, a Miyamoto-Nagai disk (Miyamoto & Nagai, 1975), and an NFW dark-matter halo (Navarro *et al.*, 1996). The calling sequence to create this model is

```
from amuse.ext.galactic_potentials import MWpotentialBovy2015
galaxy_model = MWpotentialBovy2015()
```

We initialize the proto-solar cluster by selecting a plausible starting point and investigating how the stars spread out during a 4.6 Gyr integration, without pretending that the starting point represents the actual birthplace of the Sun.

We initialize the cluster with 1000 stars with masses drawn from a Salpeter distribution between 0.1 $M_\odot$ and 10 $M_\odot$, distributed in a King (1966) model with a dimensionless depth parameter $W_0 = 3$ and a virial radius of 100 pc. We did not add stellar evolution, because with the limited number of stars only a few will be sufficiently

massive to show any noticeable effects. We evolve the cluster to an age of 4.6 Gyr using the `BHtree` direct N-body code which we bridge with the static potential using a bridge time step of 10 Myr. The resulting stellar distribution in the Galaxy is shown in Figure 9.9.

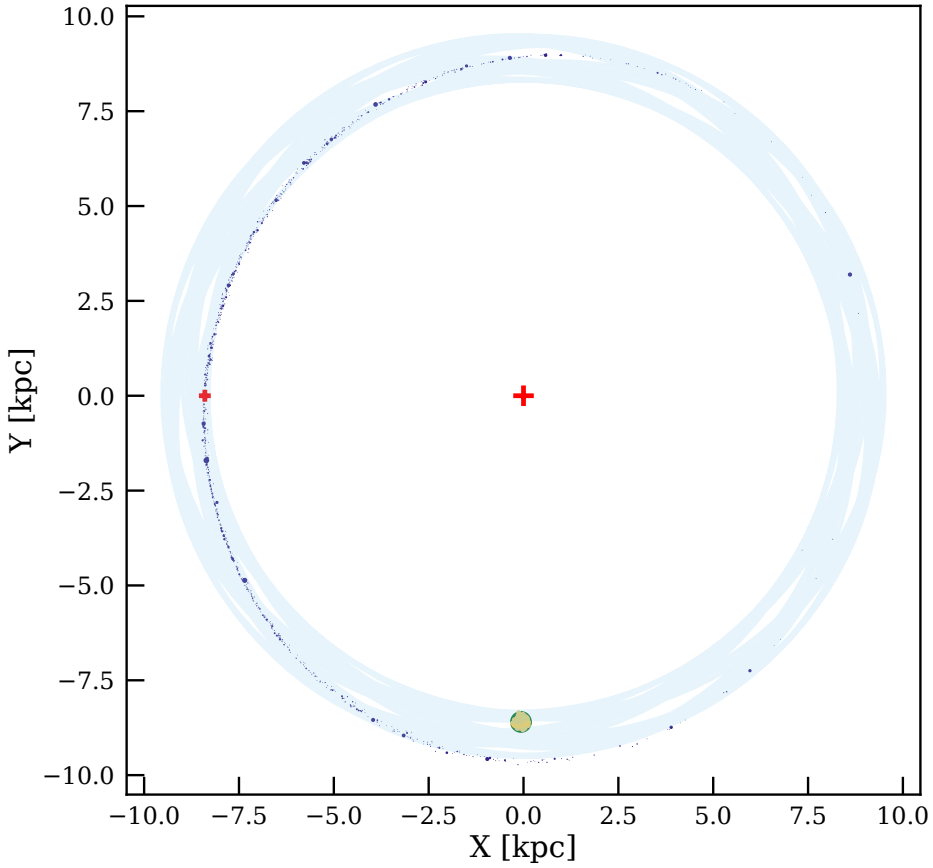

Figure 9.9: Distribution of the solar siblings (blue points) and the Sun (green dot, at left) in the Galaxy, projected in Galactic X and Y coordinates. The Galactic center is indicated by the large red cross, but no Galactic particles are shown because the Galaxy is represented here by a smooth density distribution (in contrast to Figure 9.10 where part of the galaxy is represented by giant molecular clouds and star clusters). The birth location of the Sun is indicated by the green cross at the bottom of the figure, and the initial cluster of 1000 sibling particles by the yellow points. The orbit of the Sun over the last 4.6 Gyr is indicated by the light blue line. The simulation was performed using the script `amuse.examples.solar_cluster_in_galaxy_potential`.

After having orbited the Galactic center some 27 times in 4.6 Gyr, the solar cluster has dispersed along its orbit. Based on these results, Portegies Zwart (2009) concluded that 10 to 100 siblings of the Sun could still be within a distance of 1 kpc of the current location of the Sun in the Galaxy.

9.4.1.2 Use of a rotating bridge

In order to say something substantive about the Sun's past we must adopt a more sophisticated model for the Galactic potential. A rather detailed model of the Galactic potential is available in AMUSE in the `Galaxia` package (Martínez-Barbosa *et al.*, 2016). This model consists of semi-analytic axisymmetric prescriptions for the bulge,

disk, and halo (Allen & Santillan, 1991), and a parameterization for the bar (Romero-Gómez *et al.*, 2007, 2011) that includes transient spiral arms (Cox & Gómez, 2002). The package has a lot of free parameters for which reasonable choices are needed; some of them will be tuned using data from the Gaia astrometric satellite (Gaia Collaboration *et al.*, 2016) as new data become available. Here we adopt the `Galaxia` potential for the Milky Way, for which parameters are summarized in Table A.15. The current location of the Sun is taken to be $\mathbf{x} = (-8400, 0.0, 17.0)\,\mathrm{pc}$, $\mathbf{v} = (-11.4, -232, -7.41)\,\mathrm{km/s}$ (Karachentsev & Makarov, 1996). This is a little farther away than the current best estimates of $\sim 8.0 \pm 0.3\,\mathrm{kpc}$ (Camarillo *et al.*, 2018).

We can use the `Galaxia` model in combination with bridge to integrate the Sun's orbit backward in time for 4.6 Gyr to study where it originated. The underlying `Galaxia` galaxy model is rotating, so we take Coriolis forces into account in our simulation by adopting a rotating bridge module in AMUSE (see Martínez-Barbosa *et al.*, 2016). This model is rather complex, and a simple second-order scheme for the standard bridge is insufficient to integrate the equations of motion. In the code example below we adopt a 6th-order bridge with $m = 13$ symmetric evaluations per step by setting the method option in bridge to `SPLIT_6TH_SS_M13` (see Appendix A.4). This integration scheme does an excellent job and allows us to take rather large time steps without significant loss of accuracy.

The `Galaxia` module and bridge are initialized as follows

```
from amuse.community.galaxia.interface import BarAndSpirals3D
from amuse.ext.rotating_bridge import Rotating_Bridge
from amuse.ext.composition_methods import SPLIT_6TH_SS_M13
galaxy = BarAndSpirals3D()
system = Rotating_Bridge(omega, timestep=dt_bridge, method=SPLIT_6TH_SS_M13)
```

We then initialize the Sun at its current position and velocity, and build the bridge with two components.

```
drifter = drift_without_gravity(sun_in_rotating_frame)
system.add_system(drifter, (galaxy,))
system.add_system(galaxy, ())
```

The function `drift_without_gravity` is the drifter code discussed earlier in Section 9.2.3.1, and `sun_in_rotating_frame` is the particle set that contains the Sun, corrected for the rotating frame of the `Galaxia` model. An example of how this approach works is available in `amuse.examples.integrate_sun_in_galaxy_potential`.

Adopting the current solar coordinates and integrating backward in time for 4.6 Gyr, we find the birthplace of the Sun to be $\mathbf{x} = (6641, -4953, 77.2)\,\mathrm{pc}$, with velocity $\mathbf{v} = (131.9, 197.4, 2.737)\,\mathrm{km/s}$ (right at the edge of Borg space in the Delta quadrant of the Galaxy[3]). This example should really be viewed as an illustration of how to use Bridge in conjunction with a more complex galactic potential, rather than as the ultimate method of finding the Sun's birth location. Still, the question in itself remains interesting.

[3]See http://www.startrek.com/.

9.4.1.3 Dissolution of a star cluster in a lumpy Galaxy

The previous two experiments neglected the fact that the Galaxy is lumpy, in the sense that stars and gas tend to form clusters and clouds. In the next study we add a population of clumpy objects to the Galactic potential, in the form of a number of softened point masses representing star clusters and molecular clouds, and have those objects also orbit in the Galactic potential. These objects scatter the proto-solar cluster, causing it to disperse more quickly, spreading the siblings out over a larger volume of space.

We could add clusters and clouds to our Galactic model using any of a number of published references (see e.g. Heyer & Dame, 2015), but the `GalactICs` module described earlier (see Section 4.4.5) already contains detailed prescriptions for clusters and molecular clouds, so we choose to create these objects using the `GalactICs` model. However, since `GalactICs` does not produce the same disk potential as our adopted Bovy (2015) analytic galaxy potential model (Section 9.4.1.1), we use positions from `GalactICs` but give each molecular cloud a circular velocity in the Bovy (2015) potential. Our adopted `GalactICs` model is generated with a total of 40,000 particles, 3157 of which represent the disk population, the rest follow the distribution for the Galactic halo. We use the former population of particles for the star clusters and molecular clouds, and ignore the rest. We illustrate this operation in `make_giant_molecular_clouds` in the routine `solar_cluster_in_semilive_galaxy`, which is used to generate Figure 9.10. To make the simulation somewhat more realistic we give each of these clusters a mass from a power-law mass distribution between $10^3\,M_\odot$ and $10^7\,M_\odot$ with a power-law slope of -2.3 (close to Salpeter). The massive objects are softened with a softening length of 100 pc, and are integrated using the `BHtree` tree code (Barnes & Hut, 1986).

The cluster itself has the same initial conditions and the same Galactic potential as the one adopted for Figure 9.9. Figure 9.10 presents the distribution of the stars for this second calculations at an age of 4.6 Gyr. In comparison with Figure 9.9, the siblings in the Galaxy with the massive objects are considerably more spread out due to encounters than in the case with the smooth galaxy potential. In a realistic Galaxy, the spiral arms should also be taken into account, as was done more properly in Martínez-Barbosa *et al.* (2016), although in that study (using `Galaxia`) the massive objects representing star clusters and molecular clouds were not taken into account.

9.4.1.4 Dissolution of a star cluster in a live Galaxy

We can easily replace the analytic potential used in the previous examples with a live evolving Galaxy by bridging with another gravity solver. For the Galaxy we probably don't want to use a direct N-body solver; a tree code is more appropriate. We start by generating an initial galaxy model with $N = 10000$ particles using `Halogen` (Zemp *et al.*, 2008; Zemp, 2014) as in the code example below.

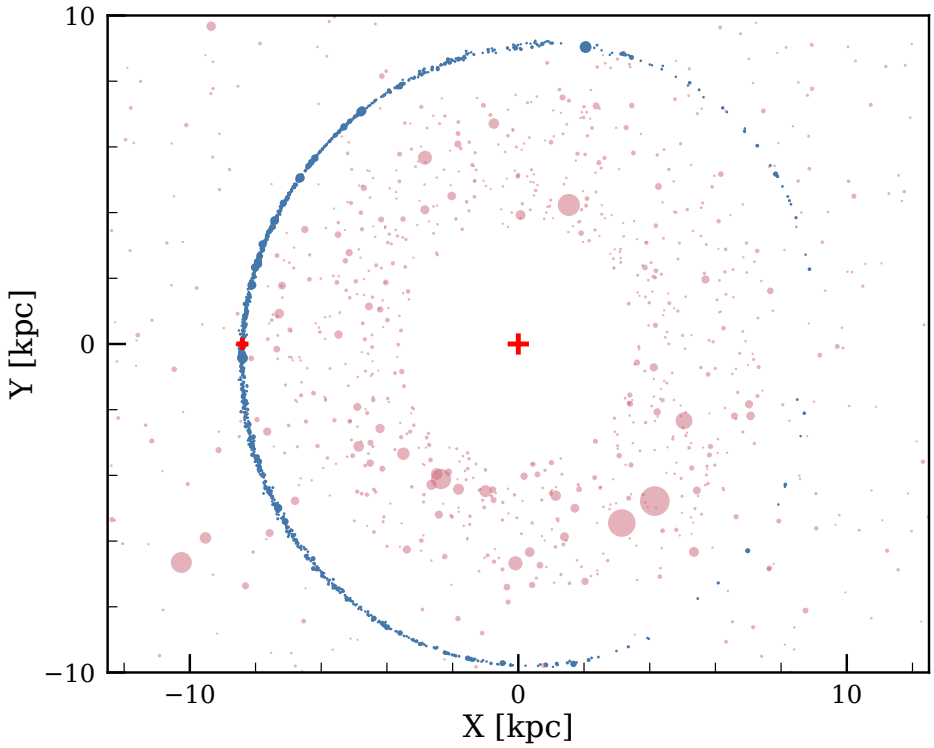

Figure 9.10: Current distribution of the solar siblings (blue bullets) relative to the Sun (green dot). The Galactic center is indicated by the red cross. No galactic particles are shown because ithe Galaxy is represented as a smooth potential, but we do show the giant molecular clouds included in the calculations (pink circles). The simulation was performed using the script `amuse.examples.solar_cluster_in_semilive_galaxy`, which should take a few minutes to run on a laptop.

```
def initialize_galaxy_model(N, converter):
    instance = Halogen(converter)
    instance.initialize_code()
    instance.parameters.alpha = 2.0
    instance.parameters.beta = 5.0
    instance.parameters.gamma = 0.0
    instance.parameters.number_of_particles = N
    instance.commit_parameters()
    instance.generate_particles()
    galaxy = instance.particles.copy()
    instance.cleanup_code()
    instance.stop()
    return galaxy
```

Note that the parameters used here are comparable, but not identical, to the settings used to generate the `halogen` panel of Figure 3.1.

We create the cluster using a $W_0 = 3$ King model with a virial radius of 3 pc, which we place at the same location as in the previous experiment. In this case we adopted $N = 1000$ stars randomly selected from a Salpeter mass function between $0.1\,\mathrm{M}_\odot$ and $10\,\mathrm{M}_\odot$, resulting in a total mass of $\sim 100\,\mathrm{M}_\odot$. We evolved the galaxy using `BHTree` and the cluster with `ph4`. The results were bridged so that the cluster affects the Galaxy

and the Galaxy the cluster:

```
gravity = bridge.Bridge(use_threading=False)
gravity.add_system(cluster_gravity, (galaxy_gravity,) )
gravity.add_system(galaxy_gravity, (cluster_gravity,) )
```

In this example we have switched off the internal threading of the bridged codes by sending `use_threading=False` as an argument in `Bridge`. Under usual circumstances, threading (or multiprocessing) is enabled in bridge, because the two bridge steps are intrinsically parallel and most computers allow the use of multiple cores simultaneously. On some older computers, however, threading produces memory conflicts which can be prevented by explicitly turning it off.

We set the bridge time step to 1/32 of the initial orbital period of the cluster around the Galactic center

```
gravity.timestep = time_orbit/32.
```

resulting in a bridge time step of $\sim 12\,\mathrm{Myr}$. The results of the calculation, after evolving for $\sim 4.6\,\mathrm{Gyr}$, are presented in Figure 9.11. We see that the siblings have been dispersed considerably compared to the smooth galaxy (Figure 9.9) or the galaxy in which we take some molecular clouds into account (Figure 9.10). Of course, the granularity of the Galactic potential, which is responsible for the large sibling dispersion evident here, depends on the details of the Galactic model and is likely far smaller than in the simulation described here. The dispersion of the solar cluster is an active field of research, and our lack of understanding the past evolution of the Galaxy seriously hinders this analysis. The consequences of the degree by which the solar cluster dispersed may have profound consequences for our ability to identify and find the lost siblings of the Sun.

9.4.2 Did the Sun originate in M67?

In Section 5.4.2 we discussed the blue stragglers in the star cluster M67. We now return to this cluster in relation to the birthplace of the Sun. In recent years the argument that the Sun originated from M67 has been made, and even though in the previous section we argued that the Sun originated from a lower-mass cluster, we investigate here the arguments for M67 as the parental cluster.

We begin this discussion with the star M67-1194, which is remarkably similar to the Sun in terms of mass and composition (Önehag *et al.*, 2011). Further dynamical analysis led Gustafsson *et al.* (2016) to conclude that the Sun might have originated in this cluster, although according to Pichardo *et al.* (2012), this conclusion is hampered by differences in the orbits of the Sun and the cluster. In addition, 982 of the stars near the cluster's turn-off have a mean age of 3.9 ± 1.2 (Xiang *et al.*, 2017), which is somewhat young compared to the Sun, and M67 is rather massive, making it unlikely that the solar system would have survived in its current form (Portegies Zwart, 2009). Nevertheless, it is interesting to test the hypothesis that the Sun originated from this cluster, and therefore we further analyze this possibility, simply because M67 is the closest massive cluster today with metallicity comparable to the Sun (Pichardo *et al.*,

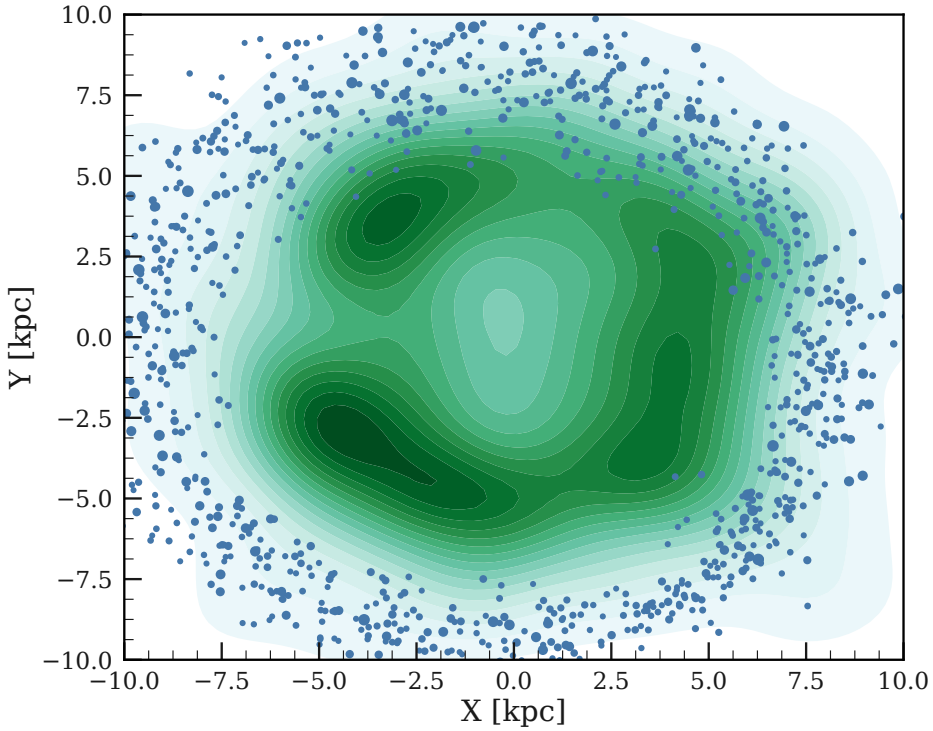

Figure 9.11: Distribution of the Solar siblings (blue bullets) in the Milky Way Galaxy (green shaded contours). We cut out the central 2 kpc from the Galaxy to keep contrast in the disk. The Sun in this simulation could be anywhere where you see a blue bullet point. The simulation was performed using the script `amuse.examples.solar_cluster_in_live_galaxy`; the figure was made with `amuse.examples.plot_solar_cluster_in_live_galaxy`.

2012).

The hypothesis that the Sun originated in M67 raises a number of interesting questions. When did the Sun escape from the cluster, and does the solar system still bear any signature of this escape? This question may be linked to the formation and morphology of the Oort cloud and other trans-Neptunian objects.

We perform a simple experiment to study when the Sun and M67 had closest approaches. Possibly a recent close approach might be associated with the moment the Sun escaped the cluster. We integrate the orbits of the Sun and the center of mass of M67 backward in time in a potential representing the Milky Way Galaxy. The last stellar encounter prior to the Sun's escaping the cluster might be a rather violent event, which could easily affect the orbits in the outer parts of the current solar system. It has therefore been argued that a single encounter, which may also have caused some long-lasting effect on the outer parts of the solar system, initiated the Sun's escape from the cluster, and the moment of escape may be identified with the closest approach between the Sun and the cluster.

We realize, of course, that the orbit of the Sun or the cluster may have been affected by a passing molecular cloud, but we ignore that in this example (although we have already demonstrated (in Section 9.4.1.3 and Section 9.4.1.4) that this makes a big difference!). We encourage the reader to experiment with a more realistic Galaxy model

and study its consequences in relation to this question in Section 9.5.2. In Section 9.4.1 we present a method by which we might include this effect in the calculation, at least statistically.

We use the potential from `Galaxia` discussed in Section 9.4.1.2 to carry out this calculation. We initialize the main script with two point masses, one for M67 and one for the Sun. For the Sun we adopt the current position and velocity used earlier in Section 9.4.1.2. M67 is located at position $(0.77, 0.0, 0.49)$ kpc and velocity $(31.92, -21.66, -8.87)$ km/s relative to the Sun (Schönrich *et al.*, 2010). Using a bridge containing both the Sun and M67 in the Galactic potential, we integrate the equations of motion backward in time for the age of the Sun, 4.56 Gyr. The age of M67 seems to be somewhat smaller, but if the sun originated from the cluster the ages must of course be about the same (we consider it unlikely that the Sun is a blue straggler). The result of this backward integration is presented in Figure 9.12. According to this calculation, the Sun and M67 passed each other at a distance of ~ 92 pc about 2.59 Myr ago.

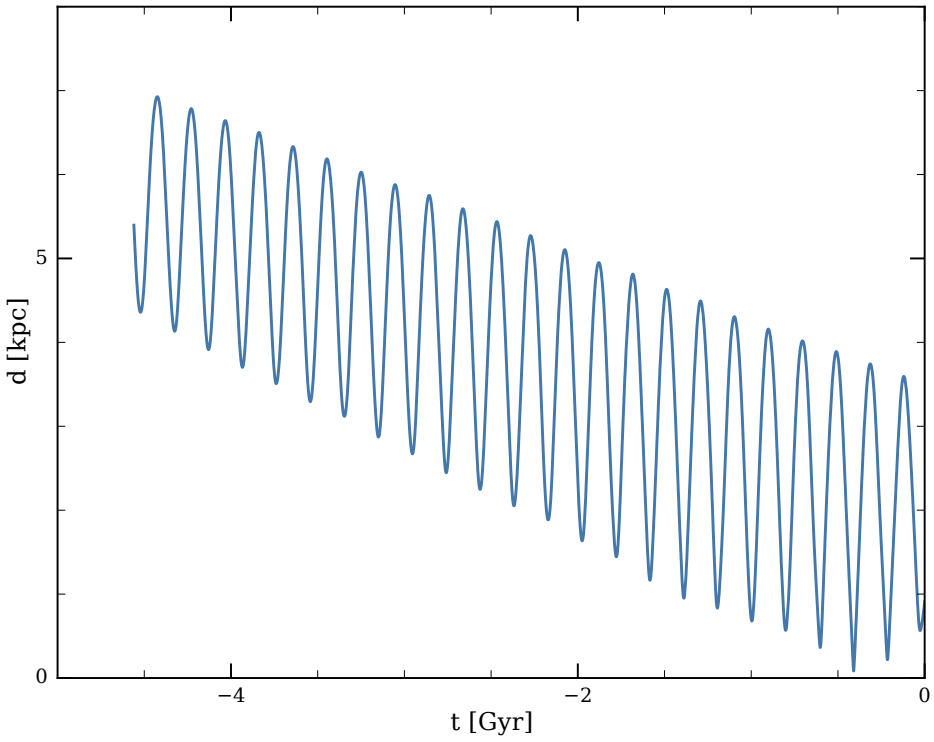

Figure 9.12: Distance between the Sun and the star cluster M67 as a function of time (negative numberz indicate the past). According to this calculation the Sun and M67 had a closest approach of ~ 92 pc some 2.59 Gyr ago. The simulation was performed using the routine in `amuse.examples.sun_and_m67_in_the_galaxy`, and took a few minutes to run.

Our assumed Galactic background potential did not evolve over the last ~ 2.59 Gyr, which is probably not realistic. Including the effects of transient spiral arms, massive molecular clouds, and star clusters surely affected the orbits over that period quite substantially (see for example Section 9.4.1.4). In Section 9.5.2 we explore when and with what velocity the Sun could have been ejected from the cluster and still end up at its current location.

9.4.3 Inspiral of a binary star into a common envelope

When two stars orbit each other, mass may flow from one to the other in a phase of Roche-lobe overflow (RLOF). Binaries are particularly vulnerable to RLOF when the more massive star ascends the giant branch. The stability of mass transfer, in terms of how rapidly mass is transferred to the companion, depends on the structure of the Roche-lobe-filling star, the response of the accreting star, and the resultant change in the orbital parameters of the binary system. RLOF in binary systems has been studied quite exhaustively, both theoretically and numerically (Webbink, 1984; Taam & Sandquist, 2000; Ivanova *et al.*, 2013).

In most triple systems RLOF probably does not proceed much differently than in a binary. For the majority of such systems, the most massive star, which evolves most rapidly, is a member of the inner binary. As a consequence, mass transfer is likely to begin in the inner binary system. The outer orbit is generally relatively wide, so mass lost from the inner binary is felt by the outer star as a slow wind. This affects the orbit of the outer star, and possibly the star itself, but probably not in a way much different than if the inner binary simply were a single star with an excess wind.

In some triples, however, this simple picture is inadequate. In those cases, the outer star is the most massive, and in a subset of these the outer orbital period is sufficiently small that mass transfer will start when the outer star ascends the giant branch. Table 9.2 presents an overview of such triples. The systems listed in Table 9.2 are indicated in Figure 9.13, where we also demonstrate that the most massive outer star experiences RLOF at some moment during the evolution.

Name	m_1	m_2 [$M_\odot$]	m_3	a_{in} [au]	e_{in}	a_{out} [au]	e_{out}	t_{RLOF} [Myr]	m_{RLOF} [$M_\odot$]	r_{Roche} [au]
τ CMa	17.8	17.8	50.0	0.076	0.0	2.49	0.285	4.3	41.5	0.95
V* CQ Dra	0.8	0.26	6.3	0.006	0.0	5.42	0.0	70.1	6.02	2.64
ξ Tau	3.2	3.1	5.5	0.133	0.0	1.23	0.15	83.7	5.50	0.42
V* V1334 Cyg	1.62	1.83	4.4	4.62	0.0	10.7	0.0	168	1.67	3.99
V* DL Vir	2.5	1.4	3.68	0.037	0.0	6.68	0.46	269	3.44	2.34
V* d Ser	2.02	1.94	2.5	0.047	0.0	1.94	0.47	799	2.46	0.62
HD 97131	1.29	0.9	1.5	0.037	0.0	0.80	0.191	2960	1.48	0.26
KIC002856960	0.24	0.22	0.76	0.008	0.0064	0.73	0.612	$> H_0$	0.72	0.12

Table 9.2: Known triples for which the outer (tertiary) star is more massive than either of the two inner stars, and the Roche radius is smaller than the maximum size of the outer star at the tip of the Asymptotic Giant Branch. Column 1 gives the common name of the triple system; columns 2–4 give the masses of the triple components; columns 5–8 give the semi-major axis and eccentricity of the inner and outer orbits; columns 9–11 give the estimated moment at which the outer (most massive) star starts to overfill its Roche lobe, the mass of that star, and its size.

RLOF in these triples is considerably different than mass transfer in an ordinary binary system. One reason is that the mass cannot simply be accreted by the inner object, but may be ejected via a slingshot effect by the rapidly orbiting inner binary. During mass transfer the inner binary also evolves, both stars may accrete some of the material delivered by the outer star, and the leftover material either forms a circumbinary disk or is ejected from the system. At the same time, orbital angular momentum

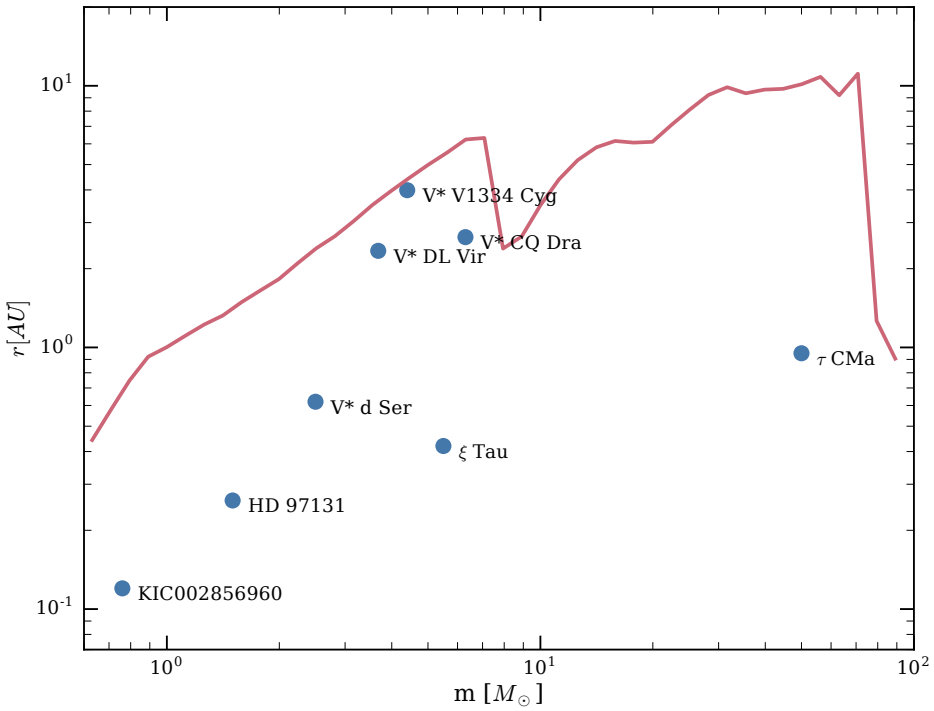

Figure 9.13: Maximum stellar radius at the tip of the asymptotic giant branch as a function of mass (red curve), calculated using `SeBa`. Blue bullets indicate the radius of the (most massive) outer component at the moment of Roche-lobe overflow, for the triple systems listed in Table 9.2. This figure was made using the script `amuse.examples.plot_rlof_triples`.

exchange may affect the eccentricity of the inner binary via to the von Zeipel-Lidov-Kozai (Lidov, 1962; Kozai, 1962) effect.

Simulating all these effects requires both an accurate gravitational solver and some way of taking the hydrodynamics of the system properly into account. The subtleties of von Zeipel-Lidov–Kozai are not well handled by the rather low-accuracy gravity solvers normally used in SPH codes. Instead, we adopt here a 4th-order Hermite solver to integrate the equations of motion of the two inner binary components and the core of the outer most star (see also Section 8.4.5). The giant envelope is modeled using an SPH code, and its core is represented as a dark-matter particle in the hydro code. The coupling between the gravity solver and the hydrodynamics code is realized using Bridge.

We adopt the observed parameters of the triple system ξ Tau as initial conditions for our simulation. We start by evolving the 5.5 $M_\odot$ primary star using EVTwin (Eggleton *et al.*, 2011; Eggleton, 1971, 2006), up to the moment when it fills its Roche lobe, at 0.42 au at an age of 83.7 Myr. The star is then converted to a hydro realization (see Section 6.4.2) with a core mass of 1.4 $M_\odot$ and placed in the triple system with the orbital parameters listed in Table 9.2. Since the stars lost virtually no mass we do not adjust the orbital parameters to take pre-RLOF mass loss into account. The orbit of the outer star is placed at the relatively low inclination of 9°; because of the von Zeipel-Lidov–Kozai effect, an inclination initially higher than $\sim 45°$ would drive the inner binary into a state of mass transfer before the outer star had a chance to undergo

RLOF.

Figure 9.14 presents the evolution of the semi-major axis of the outer orbit of the ξ Tau triple system. Since the Roche-lobe filling giant hardly loses any mass during its evolution, we simply adopt the sum of the total gas mass plus the core mass as the giant's total mass. The outer orbit shrinks within ~ 10 years from the initial 1.23 au to ~ 1.15 au after the onset of RLOF. During the same period, the eccentricity of the outer orbit decays from 0.15 to ~ 0.1.

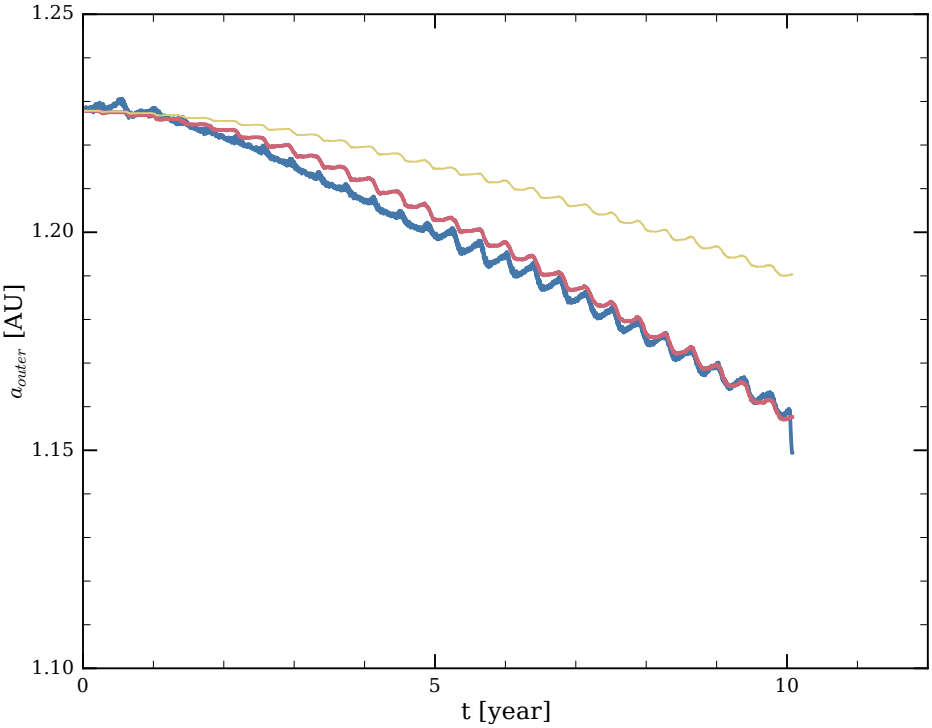

Figure 9.14: Evolution of the semi-major axis of the outer orbit in ξ Tau. In blue, we plot the result of the SPH simulations. Overplotted in red is the semi-analytic solution (Equation (9.17)) using $\eta = 6$. Two additional curves are plotted for $\eta = 4$ (yellow) and $\eta = 8$ (green). The simulation was performed using the script `amuse.examples.xi_tau_rlof`, the plotting was done with `amuse.examples.plot_xi_tau_orbital_evolution`.

We can derive an expression for the effect of mass transfer in the system, starting with a binary of primary mass m_3^0 (note here that the primary is the outer component) and secondary mass $m_{\rm in}^0 = m_1 + m_2$, and ending after mass transfer with masses m_3 and $m_{\rm in}$ for the primary and secondary stars, respectively. The degree of mass conservation is expressed by the parameter $f = (m_3 + m_{\rm in})/(m_3^0 + m_{\rm in}^0)$. Ignoring the internal angular momentum of the outer giant star and the inner binary system, we can express the evolution of the semi-major axis according to the redistribution of mass (Huang & Taam, 1990):

$$\frac{\dot{a}}{a} = -2\frac{\dot{m}_3}{m_3}\left(1 - \eta\frac{m_3}{m_{\rm in}} - \frac{1}{2}\frac{(1-\eta)m_3}{m_3 + m_{\rm in}} - f\frac{(1-\eta)m_3}{m_3 + m_{\rm in}}\right). \tag{9.17}$$

Here η is the specific angular momentum of mass leaving the system, relative to the specific angular momentum of the accreting system (in this case the center of mass of

the inner binary system). Interestingly, this is essentially the same equation (Equation (5.18)), we already saw in relation to the non-conservative evolution of semi-detached binaries in Chapter 5. Overplotted in Figure 9.14 is the evolution of the orbital separation of the outer binary in this semi-analytic model [Equation (9.17)] with $\eta = 6$, a value consistent with losing mass through the second Lagrangian point (L_2) of the outer binary system.

9.4.4 Budding planets in a protoplanetary disk

In an attempt to model the early evolution of a planetary system we simulate a young star surrounded by a gaseous disk and a number of young planets. This is a typical example in which hydrodynamics and gravitational dynamics are both relevant and operate on similar time scales, presenting an outstanding example of the power of AMUSE. The script used to perform these calculations should by now look rather familiar, except perhaps in the technique used to write composite data files (see Appendix A.2.2).

We initialize the protoplanetary system using a central star of mass $m_\star = 1.73\,\mathrm{M}_\odot$, which happens to be consistent with the mass of β Pic. This system is thought to harbor at least one rather massive planet and a gas disk, but its true composition may be considerably more complex (Winn & Fabrycky, 2015). For this simulation we limit ourselves to a collection of planets with total mass $0.01 m_\star$ and a gaseous disk of the same mass. The planets were arranged according to the Oligarchic growth model proposed by Kokubo & Ida (2002), and for which Tremaine (2015) proposed a more general scenario (see also Section 3.4.2). The disk was initialized with a Safronov–Toomre Q-parameter $Q = 1$ (see Section 6.4.1),

Our calculation had 7 planets in circular orbits ranging from a $\sim 0.4\,M_{\mathrm{Jup}}$ planet at a separation of 1.0 au to a $\sim 8.1\,M_{\mathrm{Jup}}$ planet at ~ 53 au. The gaseous disk was generated using routine `ProtoPlanetaryDisk`, as already discussed in Section 6.4.1, between 1 au and 100 au in the same plane as the planets. The equations of motions were integrated using `mercury` (Chambers & Migliorini, 1997; Chambers, 2012) and the hydrodynamics was resolved using `Fi` (Hernquist & Katz, 1989; Gerritsen & Icke, 1997; Pelupessy *et al.*, 2004). We neglected the evolution of the parent star, since we evolved the system for only 10^4 yr.

Figure 9.15 presents a snapshot of the simulation about 280 yr after the start of the simulation. The planets are represented by various colored bullets with sizes proportional to the logarithm of the planet mass. The thin disk ($Q = 1$) is perturbed by the planets, creating spiral waves. This is particularly visible at middle left, where a slightly inclined planet drives a large spiral arm in the disk. The leading arm (spiraling inwards) has its low-density wake on the outside whereas the trailing arm (going outwards) has the low-density wake on the inside. The other planets are barely visible in the image, but their presence can been seen by the structure they induce in the disk.

Although this is a rather simple demonstration of the capabilities of bridge, much more elaborate experiements including moons and other stars can easily be implemented by adjusting the example script in `amuse.examples.planetary_system`.

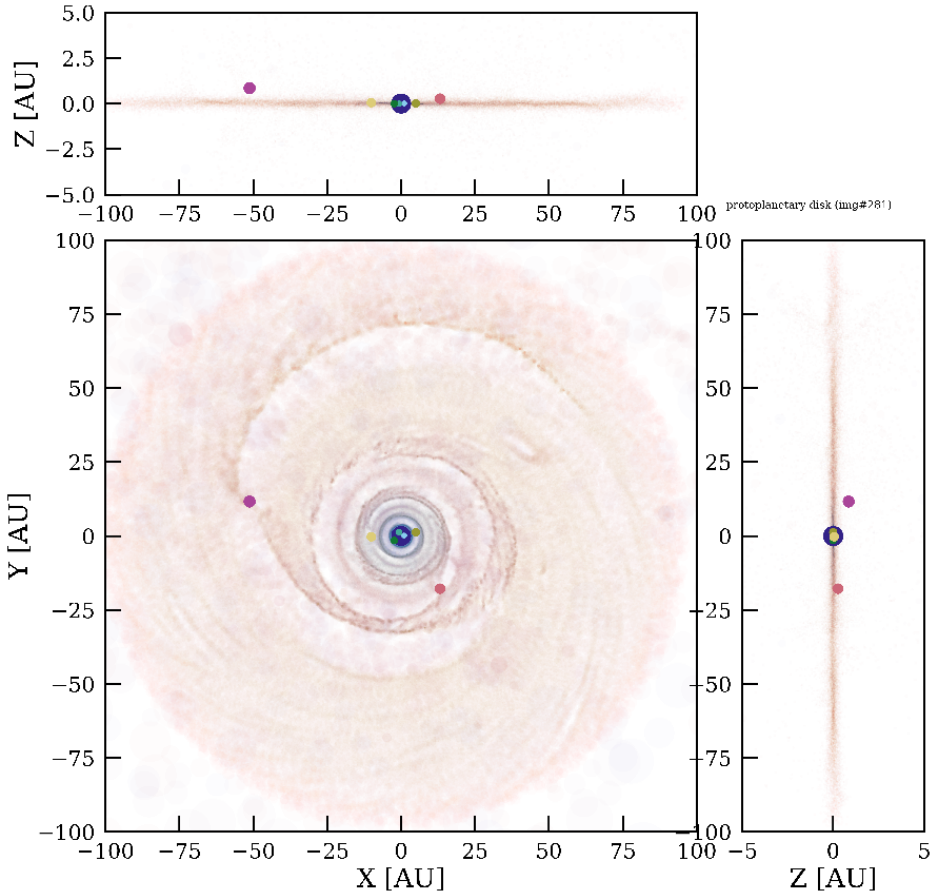

Figure 9.15: Young star with planets (bullets) and a rather flat gaseous disk. This image was made about 280 years after the start of the simulation. The planetary system was generated according to the Oligarchic growth model (see `amuse.examples.make_planets_oligarch`) with a total mass of 1% of the central star's ($1.73\,M_\odot$) mass, with the same mass in the gaseous disk (see `ProtoPlanetaryDisk`). The simulation was performed using the routine in `amuse.examples.planetary_system` (which took about a week to run); plotting was done with `amuse.examples.plot_planetary_system`.

9.4.5 Planetary systems in star clusters

Stars in star clusters are often found in pairs, or even triples or higher-order multiples. "Democratic" multiples, in which no pair of objects dominates the binding energy, tend to be unstable and quickly decay into simpler components—ultimately single stars and binaries, the fundamental building blocks of star clusters. Some higher-order hierarchies can survive for a very long time, although eventually most of those also dissolve. The realization that many stars are born with multiple planets (and smaller objects) demands a numerical treatment of hierarchical systems. (We note in passing that a stellar distribution around a supermassive black hole is in many ways a similar numerical problem.)

So far we have only discussed binaries in our discussion of stellar evolution, and how to manage encounters in star clusters in Sections 8.7.1 and 8.7.2, Here we briefly discuss the ongoing development of how to handle rich hierarchical systems of stars, planets and maybe even planetesimals. The module discussed in Section 8.7.2 is intended for

simulating star clusters with a rich population of binaries, whereas the `Nemesis` module described here is meant for integrating planetary systems inside a star cluster.

The relatively short orbital periods of planetary systems with respect to the crossing time of a star cluster gives rise to a number of interesting numerical challenges. One of these is wide range in time scales, which can be days for a planet orbiting a star compared to a million years for a star orbiting the center of mass of a cluster. This wide separation in scales allows us to treat the planetary system differentely than the star cluster, often using a different code. The bridge method is ideally suited for such a hierarchical structure.

In `Nemesis` (see Section 10.1.2) each planetary system is integrated with its own individual integrator. When several stars or planetary systems approach each other within a predefined limit, both systems are merged and the integration is continued using a single integrator. If a multi-body system dissolves, single objects are picked up by the global N-body integrator whereas multi-body subsystems continue to be integrated with individual solvers. Integrators in this method are dynamically created and deleted at runtime. `Nemesis` is fast, efficient, parallel, and effective in addressing multi-planetary systems in a star cluster. At the time of writing the `Nemesis` module is not yet avaiable in AMUSE, as we are still in the testing phase (see Figure 9.16). We can say, however, that the method seems to work as expected, and that `Nemesis` will soon become part of the AMUSE code base.

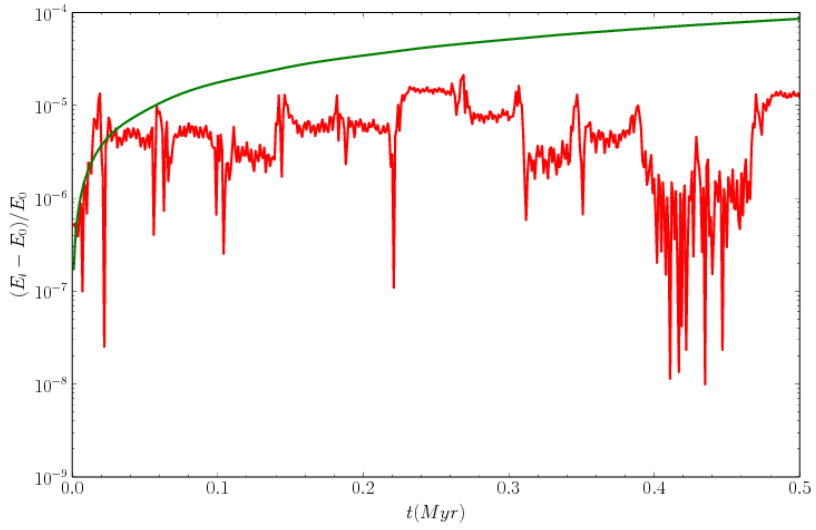

Figure 9.16: Evolution of the numerical integration error in the energy of the solar system when orbited by second $1\,M_\odot$ star. In this example the other star (allegedly called Nemesis) orbits the solar system in the ecliptic plane with a semi-major axis of 1 pc and an eccentricity of 0.6. The green curve gives the energy error when integrated using `Hermite`; the red curve give the result of the Nemesis module.

9.5 Assignments

9.5.1 Drift with gravity code

In Section 9.3.1 we discussed a small experiment in which we integrated the equations of motion for the Sun, Earth, and Moon using `Bridge`. In that example we employed three incarnations of `ph4` to integrate a single Sun, a single Earth and a single Moon, and coupled these three codes using multiple bridges. This was a little excessive, and it would have been much better to use the drifter code in Code example 9.5 instead of the direct N-body codes. Rewrite Code example 9.6, Code example 9.7 and Code example 9.8 using the drifter code from Code example 9.5, and compare them with a small script in which all the particles are integrated using the direct N-body code. Which of the various methods would you prefer, based on reliability, speed, and accuracy?

9.5.2 How did the Sun escape from M67?

If the Sun escaped from the star cluster M67, as discussed in Section 9.4.2, this might have happened quite a while ago, when the cluster and the Sun had a closest approach. In the small calculation performed earlier, we adopted a simple analytic potential for the Galactic background, launched the Sun and M67 from their current locations, and integrated backwards in time for 4.56 Gyr. According to this calculation, at closest approach, the distance between the Sun and the M67 was 92 pc. However, not mentioned in Section 9.4.2 is that the relative velocity between Sun and the cluster at that moment was ~ 48 km/s, which is rather high. An encounter with another cluster member could have ejected the Sun, but this would probably have led to a velocity of the order of the cluster's escape speed, which is an order of magnitude smaller. An encounter that propelled the solar system to a speed of almost 50 km/s would have profound consequences for the further evolution of most of the planets orbiting the Sun.

In this assignment we recognize that the current coordinates of M67 are not precisely known, and ask if we were allowed some freedom, would it be possible to find a better match between the trajectories of the Sun and the cluster? First repeat the calculation in Section 9.4.2 and confirm the earlier result. Then allow the current position and velocity of M67 relative to the solar system to vary within the observed uncertainty, and search for the closest historical distance and relative velocity between the two point masses in the Galactic potential. The underlying assumption is that the Sun was ejected from M67 and that its orbit has not changed appreciably since then.

We could search parameter space by hand, but this is a rather painful process. A better method would be to adopt a Markov-Chain Monte Carlo (MCMC) approach using the Metropolis-Hastings algorithm (Metropolis *et al.*, 1953; Hastings, 1970) to find the optimum solution. This algorithm works quite well for searching a large parameter space. There are more efficient alternatives (Feroz *et al.*, 2009), but MCMC tends to find solutions close to the optimum quite effectively and it is rewarding to implement by yourself.

The Metropolis-Hastings algorithm works as follows.

1. Perform a calculation with some predetermined initial set of parameters, like the Sun's current location and velocity in the Galaxy and that of M67.

2. Integrate both objects backwards in time in the Galactic potential for 4.6 Gyr and measure the minimum "distance" (in position r and velocity v) between the Sun and M67. Let's call this phase-space distance d. In order to be able to add distance and velocity we have to normalize them, in which case one could define d by $d^2 = (r/r_{\rm min})^2 + (v/v_{\rm min})^2$, where $r_{\rm min}$ and $v_{\rm min}$ are the minimum joint values found for r and v. After the first calculations $d^2 = 2$.

3. Introduce a small change to the initial position and velocity of M67 within the observational uncertainty, or simply adopt a variation of ~ 100 pc in position and ~ 1 km/s in velocity. You could use a more formal approach from the literature, but this procedure will give a reasonable solution; we can later return to this assumption to see if the final solution is reasonable.

4. Repeat the calculation with the introduced variation in the position and velocity of M67, and measure the new minimum phase-space distance, which we call d'. If $d' > d$ the Sun passed the orbit of M67 at a larger phase-space distance, but if $d' < d$ the new solution is better.

5. Now draw a random number X on the interval $[0, 1]$. If $X < \exp(-(d' - d)/d)$ we accept the new initial conditions as the best coordinates for M67, and we update $r_{\rm min}$ and $v_{\rm min}$ accordingly. Otherwise we keep the previous best initial conditions.

6. Repeat this procedure (from step 3 to step 5) until you are happy with the results. While performing the iterative procedure you may want to reduce the uncertainty. It is generally best to do this slowly and preferably only when you adopt the new initial realization. You will find that the quality of the calculation improves with the number of iterations. The rate at which it improves depends on your choices for the variations in position and velocity introduced in each new calculation. It may be rewarding to experiment with this to find the most efficiently converging Markov chain.

Instead of writing your own MCMC code, you could adopt a package, such as `emcee` (Foreman-Mackey *et al.*, 2013) which, due to its Python interface, works naturally with AMUSE.

Address the following questions:

- What is the minimum distance and relative velocity you find for M67 and the solar system?

- Is this a realistic solution? Take into account here that the orbital speed of Neptune is ~ 5.5 km/s, that the solar system probably already had the planet by the time it left the cluster, and Neptune still orbits the Sun.

- What difference does it make if you change the potential of the Galaxy, change the number of spiral arms using the `BarAndSpirals3D()` routine from `Galaxia` as discussed in Section 9.4.1.2, or introduce giant molecular clouds (see Section 9.4.1)?

- Instead of searching for the optimal position and velocity of the cluster, you could relax the assumptions about the Galaxy, such as the mass of the bulge and bar, the pitch angle of the spiral arms or their rotation speed. One advantage of the Metropolis-Hastings algorithm is that it is rather agnostic about the parameters you change so long as you define a proper phase-space distance on which to optimize.

9.5.3 Half-treecode, half-direct

We want to study a large system of self-gravitating particles initially distributed in a virialized Plummer (1911) sphere, with a stellar mass function running from $0.1\,\mathrm{M}_\odot$ to $100\,\mathrm{M}_\odot$. We have insufficient computer resources to perform the entire calculation using a direct N^2 method, but we require a precision that exceeds that of a standard treecode. We therefore decide to create a hybrid *split method* in which we combine a treecode with a direct N-body code by interfacing the two codes using `Bridge` within the AMUSE framework.

We divide the stars among the two codes, where the treecode handles the low-mass stars (those with $m < m_{\rm cut}$) and the direct N-body code takes care of the high-mass stars (with $m \gtrsim m_{\rm cut}$). In order to optimize the simulation, we will study the behavior of the hybrid code as a function of the mass cut $m_{\rm cut}$ which determines which particles are treated by which code. Although we will perform the numerical experiments with units, we will not take stellar evolution into account because we are currently only interested in validating the proposed hybrid method.

1. Why do we split the particle set in mass rather than, for example, in distance from the center of mass or as a function of local density?

2. Estimate the bridge time step relative to the typical orbital timescale you will use for your calculation.

3. Write the hybrid script and give it an interface so that you can run it from the command line.

Initialize the system with $N = 10^4$ stars with a virial radius of $r = 3\,\mathrm{pc}$, and integrate it for $10\,\mathrm{Myr}$. Perform the following three runs and store the half-mass radius, the core radius, and the total kinetic and potential energies at a time resolution of $0.1\,\mathrm{Myr}$:

- all stars in the treecode;

- all stars in the direct code;

- split code, with $m_{\rm cut} = 1.0\,\mathrm{M}_\odot$.

Make the following two plots

- the relative energy error compared to the initial energy $((E - E_0)/E_0)$ as a function of time,
- the half-mass and core radii as functions of time.

Also make a table of the wall-clock times of these three runs.

Discuss your findings, and say whether or not running low-mass stars with a tree code and high-mass stars with a direct code is an advantageous strategy.

9.5.4 The accreting black hole in HLX-1

HLX-1 is a variable and bright ($\gtrsim 10^{42}$ erg/s) x-ray source in the direction of the spiral galaxy ESO 243-49 (Farrell *et al.*, 2009). The reasons for its repeated outbursts and the nature of the x-ray source remain elusive (Webb *et al.*, 2017). Figure 9.17 shows a portion of the x-ray counts reported by the SWIFT XRT instrument from Fig. 3 of (Titarchuk & Seifina, 2016).

Observations support the hypothesis that the x-rays are produced by a black hole of mass $10^3 - 10^5\,\mathrm{M}_\odot$ that is orbited by a massive star (Webb *et al.*, 2012). The companion star is thought to be evolved, so that it fills its Roche lobe near pericenter, but at apocenter no mass is transferred and therefore no x-rays are produced. We can make a very simple model to test this hypothesis by integrating the orbit of a $20\,\mathrm{M}_\odot$ star around a black hole. In Figure 9.17 we present the results of such a simple calculation in which the giant donor orbits a $10^3\,\mathrm{M}_\odot$ black hole in a 370 day orbit. To ensure that the donor overflows its Roche lobe near pericenter we adopt an eccentricity of 0.95. The x-ray count rate is assumed to be proportional to the degree to which the $20\,\mathrm{M}_\odot$ overfills its Roche lobe, which only happens near pericenter.

The counts calculated in the simple model do not align with the observed counts, although a number of features of the observations are shared by the model calculation. Tidal effects, as well as the viscous evolution of the donor star, probably contribute to the variations observed in the x-ray outburst period. In this assignment we will try to improve the simple model used to produce Figure 9.17. We adopt the `MESA` (Paxton *et al.*, 2010, 2011) stellar evolution package with the Huayno N-body integrator (Pelupessy *et al.*, 2012). We evolve a $20\,\mathrm{M}_\odot$ star for 9.0 Myr, by which time it has ascended the giant branch. We start the simulation when the star has lost $1.54\,\mathrm{M}_\odot$ as a stellar wind, and its radius is 4.19 au, sufficient to fill its Roche lobe at pericenter for the adopted binary period.

There are two basic approaches we can follow here—model the tidal evolution of the donor star in the black-hole field, or use a smoothed-particle hydrodynamics code to resolve the donor. For tidal evolution we recommend using the `tidal_interaction` routine from the `stellar_tidal_evolution` package in `amuse.ext`. This prescription follows the tidal evolution of planets and stars according to the model by Hansen (2010), which is based on the theory of equilibrium tidal models by Eggleton *et al.* (1998).

To use an SPH code to resolve the donor star in its orbit around the black hole, we convert the outer layers of the donor star to an SPH realization using at least 1000

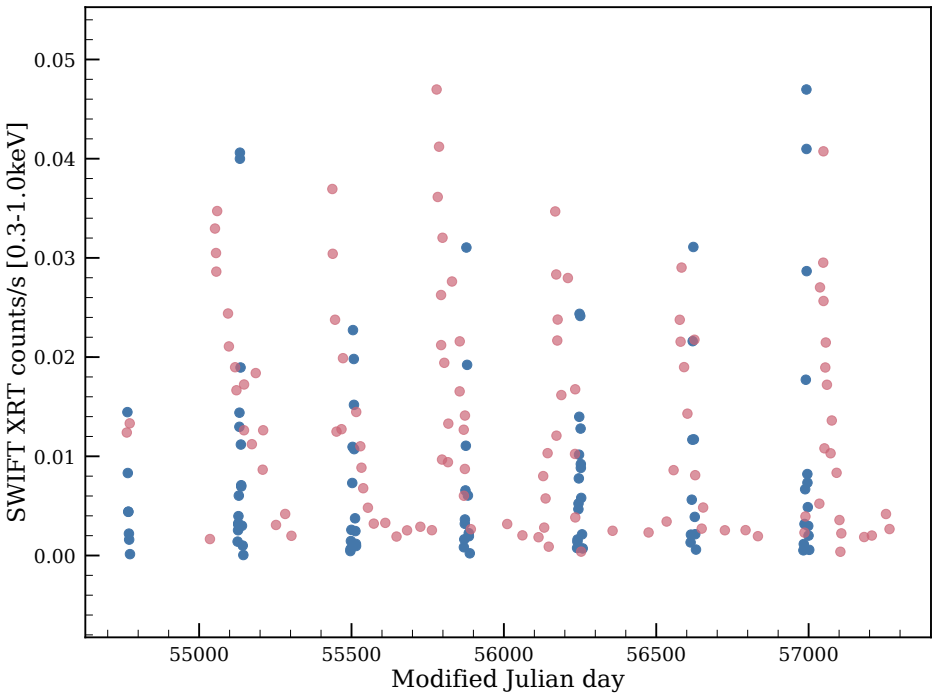

Figure 9.17: X-ray counts of the source HLX-1 as observed by SWIFT XRT in the 0.3–1.0 keV band (Titarchuk & Seifina, 2016) (red bullets). Overplotted are the results of the simple model calculation described in the text (blue bullets).

particles, with the central portion of the star represented by a single massive particle. This can be realized by generating an SPH realization of a star in which a core mass is specified (see Section 8.5.4.1 and in particular Code example 8.11, where we discussed how to convert a stellar evolution model to an SPH realization). Code example 9.15 shows how a star can be evolved to the giant stage and subsequently converted to a SPH realization with a single particle representing its core. We used the same method in Section 9.4.3 to generate the giant star with a core in order to simulate the triple system Xi Tau. In Code example 9.15 we use the argument `with_core_partricle` to indicate that the stellar model has a core which should be returned in a separate particle set.

A bridge can then be built with the gravity code to integrate the black hole and the donor's core, using the SPH code to resolve the stellar envelope. Allow the black hole to accrete using a sink particle, and measure, as a function of time, the amount of material accreted by the black hole. Integrate the system for half a dozen orbits, as demonstrated in Figure 9.17, and plot the accretion rate of the black hole together with the observational data.

Which of the two models, tidal evolution or detailed hydrodynamics, gives a better comparison with observations?

```
    stellar = Mesa()
    stellar.particles.add_particle(Particle(mass=o.mass))
    XiTau = stellar.particles[0]
    while XiTau.radius <= o.radius:
        print(
            f"Time={stellar.model_time.in_(units.Myr)} "
            f"{stellar.particles.radius.in_(units.AU)} "
            f"{stellar.particles.stellar_type}"
        )
        stellar.evolve_model()

    target_core_mass = XiTau.core_mass
    print(XiTau)
    print("Core mass:", XiTau.core_mass.in_(units.MSun))
    sph_model = convert_stellar_model_to_sph(
        XiTau,
        o.Nsph,
        with_core_particle=True,
        target_core_mass=target_core_mass,
        do_store_composition=False,
        base_grid_options=dict(type="fcc")
    )
    core_particle = sph_model.core_particle.as_set()
    gas_particles = sph_model.gas_particles

    print(f"Ngas={len(gas_particles)} Ncore={core_particle}")
    write_set_to_file(
        core_particle, "Hydro_PrimaryStar_XiTau.amuse", "amuse",
        append_to_file=False
    )
    write_set_to_file(gas_particles, "Hydro_PrimaryStar_XiTau.amuse", "amuse")
    stellar.stop()
```

Code example 9.15: Evolving a star and turning it into a SPH realization with a single particle representing the stellar core.

9.5.5 The formation of the widest binary stars

In 1915 Robert Innes discovered that the binary system α Centauri is orbited by a third single star (Innes, 1915). The discovery was made by examining photographic plates of the region around α Centauri using a blink microscope to search for faint stars with large proper motion (Innes, 1915). It was not until 1917 that Innes (1917) referred to this star as Proxima Centauri: "If this small star had a name it would be convenient—it is therefore suggested that it should be referred to as Proxima Centaurus."

Today the orbit of Proxima Centauri is well constrained to have semi-major axis $8.7^{+0.7}_{-0.4} \times 10^3$ au and eccentricity $0.50^{+0.08}_{-0.09}$ (Kervella *et al.*, 2017). The inner α Centauri binary has a semi-major axis of about 23.7 au and an eccentricity of 0.524 ± 0.0011; the three stars have masses $1.105 \pm 0.0070\,\mathrm{M}_\odot$, $0.934 \pm 0.0061\,\mathrm{M}_\odot$ and $\sim 0.123\,\mathrm{M}_\odot$ (Pourbaix *et al.*, 2002). This very wide orbit is not explained from standard star (and binary) formation theories. An interesting alternative for the formation was proposed by Reipurth & Mikkola (2012). They argue that three stars embedded in a gas cloud can find each other and engage in a triple resonant interaction. The result of the interaction

is the ejection of one of the stars, but due to the presence of the surrounding gas the star does not escape but finds itself parked in a wide orbit around the binary system.

The calculations performed in Reipurth & Mikkola (2012) employed a direct N-body code which in AMUSE is called `Mikkola`. In order to perform the calculations, Reipurth & Mikkola (2012) added a smooth background potential representing the gas. They concluded that the existence of extremely hierarchical systems is a natural consequence of resonant interactions embedded in a primordial gas reservoir. Fundamental to this formation channel is the interaction with the background gas potential, accretion onto the stars, and stellar gravitational interactions. With AMUSE we can relatively easily reproduce their results, but we can also include the actual background gas, rather than a static background potential.

In Section 9.5.3 we had an assignment in which we bridged two gravity codes, one for the low-mass "background" stars and the other for the more massive stellar population. Here we will replace the background population with a gas distribution, simulated using an SPH code.

1. Adopt a mass of $0.1\,\mathrm{M}_\odot$ for each of the three stars and distribute them in a Plummer sphere with a characteristic radius of 400 au. Reipurth & Mikkola (2012) adopted a mean separation between 40 au and 400 au.

2. Add an SPH particle representation to the system, centered on the center of mass of the three-body system, with a mass of $10\,\mathrm{M}_\odot$ and characteristic radius 7500 au.

3. Reipurth & Mikkola (2012) allowed the three stars to accrete from the ambient gas. We can take accretion onto the three stars into account by having a sink particle move along with the stars (see Section 6.3.2). Stop the simulation after 100 Myr and study the resulting triple system.

Reipurth & Mikkola (2012) argued that the resulting projected orbital separation s matches the power law (see their Fig. S1)

$$f(s)ds \sim s^{-1.6}ds. \tag{9.18}$$

Perform a number of simulations (Reipurth & Mikkola, 2012; performed a total of 180,000) with randomized initial realizations from the above initial conditions, and compare their projected separation with the distribution suggested by Reipurth & Mikkola (2012).

A key aspect of this model is that the three embryonic stars are initially identical, but are forced to compete for gas to accrete through dynamical interactions. The consequential change in mass subsequently feeds back into their orbital evolution, resulting in the power-law distribution in orbital separation. Note however, that the obtained $s^{1.6}$ power-law is in reasonable agreement with observations (or maybe a bit steeper, see Reipurth & Mikkola, 2012).

Repeat the experiment using 4 stars rather than 3, each with a mass of $0.1\,\mathrm{M}_\odot$, adopting the same initial conditions as before, and including the sink particles. Answer the following two questions:

- What is the fate of the fourth star?

- What are the distributions of the final masses of the four stars?
- Do you expect that the inner binary contains the two most massive stars, as in the observed Alpha-Proxima Centauri triple system.

9.5.6 Photoevaporating disk in binary system

In Section 10.1.3 we discuss the possibility of coupling multiple codes into a self-scheduling organized fashion. The described method, `venice`, allows multiple codes to be coupled on their own internal temporal resolved and spatially resolved scales. This leads to AMUSE to address intricate intricate multi-scale and multi-phyiscs problems.

One such a problem is the photoevaporation and gravitational perturbation of a circumstellar disk around a low-mass star in an eccentric orbit around a massive companion star. Such a disk is hydrodynamically affected by the gravity of the companion, while being photoevaporated. In AMUSE two methods can be effective in addressing this problem. It is possible to resolve the disk using a hydrodynamics solver together with the gravity using an N-body solver and the radiation with a radiative transfer code. The other way would be using the combination of `Vader` (see Appendix B.2.3), as a disk-evolution code, with a Kepler solver together with the FRIED grid (see Appendix B.2.4). Both methods allow you to study various aspects of such a problem. In this assignment we are going to try both methods.

The key aspect for the evolution of an irradiated and gravitationally influenced disk around a low-mass star is the truncation, evaporation, and structural changes in the disk due to the other star. The first method, in principle, incorporates all these effects, whereas the more parameterized method only informs us about the disk's photo-evaporation.

Write a script to address the coupled problem by bridging an SPH code for the disk with a radiative transfer code, a stellar evolution code, and a direct N-body solver for the mutual gravity.

Write a second script, using `venice`, where the gravity is addressed with a Kepler solver, the disk using `vader`, the stellar evolution with the appropriate solver, and the radiative interaction with the FRIED grid.

Answer the following questions:

- To what extent is the disk gravitationally affected by the companion star?
- To what extent is the disk gravitationally affected by the radiative transfer?
- What can you say about the difference is speed when comparing the two scripts?
- Which of the two approaches is most satisfactory for addressing the problem's key questions?

Bibliography

Adams, D.N. 1979. *The Hitchhiker's Guide to the Galaxy.* Pan Books.

Allen, Christine, & Santillan, Alfredo. 1991. An improved model of the galactic mass distribution for orbit computations. *Revista Mexicana de Astronomia y Astrofisica* , **22**(Oct.), 255.

Barnes, Josh, & Hut, Piet. 1986. A hierarchical O(N log N) force-calculation algorithm. *Nat* , **324**(6096), 446–449.

Bottke, Jr., W. F., Vokrouhlický, D., Rubincam, D. P., & Broz, M. 2002. The Effect of Yarkovsky Thermal Forces on the Dynamical Evolution of Asteroids and Meteoroids. *Pages 395–408 of:* Bottke, Jr., W. F., Cellino, A., Paolicchi, P., & Binzel, R. P. (eds), *Asteroids III*.

Bovy, Jo. 2015. galpy: A python Library for Galactic Dynamics. *ApJS* , **216**(2), 29.

Camarillo, Tia, Mathur, Varun, Mitchell, Tyler, & Ratra, Bharat. 2018. Median Statistics Estimate of the Distance to the Galactic Center. *PASP* , **130**(984), 024101.

Chambers, J. E., & Migliorini, F. 1997 (July). Mercury - A New Software Package for Orbital Integrations. *Page 27.06 of: AAS/Division for Planetary Sciences Meeting Abstracts #29*. AAS/Division for Planetary Sciences Meeting Abstracts, vol. 29.

Chambers, John E. 2012 (Jan.). *Mercury: A software package for orbital dynamics*. Astrophysics Source Code Library, record ascl:1201.008.

Cooder, R. 1972. *Crow Black Chicken*.

Cox, Donald P., & Gómez, Gilberto C. 2002. Analytical Expressions for Spiral Arm Gravitational Potential and Density. *ApJS* , **142**(2), 261–267.

Eggleton, P. P., Tout, Christopher, Pols, Onno, Izzard, Rob, Eldridge, John, Lesaffre, Pierre, Stancliffe, Richard, Church, Ross, & Lau, Herbert. 2011 (July). *STARS: A Stellar Evolution Code*. Astrophysics Source Code Library, record ascl:1107.008.

Eggleton, Peter. 2006. *Evolutionary Processes in Binary and Multiple Stars*.

Eggleton, Peter P. 1971. The evolution of low mass stars. *MNRAS* , **151**(Jan.), 351.

Eggleton, Peter P., Kiseleva, Ludmila G., & Hut, Piet. 1998. The Equilibrium Tide Model for Tidal Friction. *ApJ* , **499**(2), 853–870.

Farrell, Sean A., Webb, Natalie A., Barret, Didier, Godet, Olivier, & Rodrigues, Joana M. 2009. An intermediate-mass black hole of over 500 solar masses in the galaxy ESO243-49. *Nat* , **460**(7251), 73–75.

Feroz, F., Hobson, M. P., & Bridges, M. 2009. MULTINEST: an efficient and robust Bayesian inference tool for cosmology and particle physics. *MNRAS* , **398**(4), 1601–1614.

Foreman-Mackey, Daniel, Hogg, David W., Lang, Dustin, & Goodman, Jonathan. 2013. emcee: The MCMC Hammer. *PASP* , **125**(925), 306.

Fujii, Michiko, Iwasawa, Masaki, Funato, Yoko, & Makino, Junichiro. 2007. BRIDGE: A Direct-Tree Hybrid N-Body Algorithm for Fully Self-Consistent Simulations of Star Clusters and Their Parent Galaxies. *Publ. Astr. Soc. Japan* , **59**(Dec.), 1095.

Gaia Collaboration, Brown, A. G. A., Vallenari, A., Prusti, T., de Bruijne, J. H. J., Mignard, F., Drimmel, R., Babusiaux, C., Bailer-Jones, C. A. L., Bastian, U., & et al. 2016. Gaia Data Release 1. Summary of the astrometric, photometric, and survey properties. *A&A* , **595**(Nov.), A2.

Gerritsen, J. P. E., & Icke, V. 1997. Star formation in N-body simulations. I. The impact of the stellar ultraviolet radiation on star formation. *A&A* , **325**(Sept.), 972–986.

Gustafsson, Bengt, Church, Ross P., Davies, Melvyn B., & Rickman, Hans. 2016. Gravitational scattering of stars and clusters and the heating of the Galactic disk. *A&A* , **593**(Sept.), A85.

Hansen, Brad M. S. 2010. Calibration of Equilibrium Tide Theory for Extrasolar Planet Systems. *ApJ* , **723**(1), 285–299.

Harfst, S., Portegies Zwart, S., & Stolte, A. 2010. Reconstructing the Arches cluster - I. Constraining the initial conditions. *MNRAS* , **409**(2), 628–638.

Hastings, W. K. 1970. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika*, **57**(1), 97–109.

Hernquist, Lars, & Katz, Neal. 1989. TREESPH: A Unification of SPH with the Hierarchical Tree Method. *ApJS* , **70**(June), 419.

Heyer, Mark, & Dame, T. M. 2015. Molecular Clouds in the Milky Way. *ARA&A* , **53**(Aug.), 583–629.

Huang, R. Q., & Taam, R. E. 1990. The non-conservative evolution of massive binary systems. *A&A* , **236**(1), 107–116.

Hut, Piet, Makino, Jun, & McMillan, Steve. 1995. Building a Better Leapfrog. *ApJL* , **443**(Apr.), L93.

Innes, R. T. A. 1915. A Faint Star of Large Proper Motion. *Circular of the Union Observatory Johannesburg*, **30**(Oct.), 235–236.

Innes, R. T. A. 1917. Parallax of the Faint Proper Motion Star Near Alpha of Centaurus. 1900. R.A. 14 h 22m 55s.-0s 6t. Dec-62&deg 15'2 0'8 t. *Circular of the Union Observatory Johannesburg*, **40**(Sept.), 331–336.

Ivanova, N., Justham, S., Chen, X., De Marco, O., Fryer, C. L., Gaburov, E., Ge, H., Glebbeek, E., Han, Z., Li, X. D., Lu, G., Marsh, T., Podsiadlowski, P., Potter, A., Soker, N., Taam, R., Tauris, T. M., van den Heuvel, E. P. J., & Webbink, R. F. 2013. Common envelope evolution: where we stand and how we can move forward. *A&A Review* , **21**(Feb.), 59.

Karachentsev, I. D., & Makarov, D. I. 1996. Orbital velocity of the Sun and the apex of the Galactic center. *Astronomy Letters*, **22**(4), 455–458.

Kervella, P., Thévenin, F., & Lovis, C. 2017. Proxima's orbit around α Centauri. *A&A* , **598**(Feb.), L7.

King, Ivan R. 1966. The structure of star clusters. III. Some simple dynamical models. *AJ* , **71**(Feb.), 64.

Kokubo, Eiichiro, & Ida, Shigeru. 2002. Formation of Protoplanet Systems and Diversity of Planetary Systems. *The Astrophysical Journal*, **581**(1), 666.

Kozai, Yoshihide. 1962. Secular perturbations of asteroids with high inclination and eccentricity. *AJ* , **67**(Nov.), 591–598.

Landau, Lev Davidovich, & Lifshitz, E. M. 1969. *Mechanics*.

Lidov, M. L. 1962. The evolution of orbits of artificial satellites of planets under the action of gravitational perturbations of external bodies. Planet. Space Sci., **9**(10), 719–759.

Makino, Junichiro. 1991. Optimal Order and Time-Step Criterion for Aarseth-Type N-Body Integrators. *ApJ* , **369**(Mar.), 200.

Martínez-Barbosa, C. A., Brown, A. G. A., Boekholt, T., Portegies Zwart, S., Antiche, E., & Antoja, T. 2016. The evolution of the Sun's birth cluster and the search for

the solar siblings with Gaia. *MNRAS* , **457**(1), 1062–1075.
McMillan, Stephen L. W., & Portegies Zwart, Simon F. 2003. The Fate of Star Clusters near the Galactic Center. I. Analytic Considerations. *ApJ* , **596**(1), 314–322.
Metropolis, Nicholas, Rosenbluth, Arianna W., Rosenbluth, Marshall N., Teller, Augusta H., & Teller, Edward. 1953. Equation of State Calculations by Fast Computing Machines. *J. Comp. phys.* , **21**(6), 1087–1092.
Mezger, Peter G., Duschl, Wolfgang J., & Zylka, Robert. 1996. The Galactic Center: a laboratory for AGN? *A&A Review* , **7**(4), 289–388.
Mishurov, Yu. N., & Acharova, I. A. 2011. Is it possible to reveal the lost siblings of the Sun? *MNRAS* , **412**(3), 1771–1777.
Miyamoto, M., & Nagai, R. 1975. Three-dimensional models for the distribution of mass in galaxies. *Publ. Astr. Soc. Japan* , **27**(Jan.), 533–543.
Navarro, Julio F., Frenk, Carlos S., & White, Simon D. M. 1996. The Structure of Cold Dark Matter Halos. *ApJ* , **462**(May), 563.
Önehag, A., Korn, A., Gustafsson, B., Stempels, E., & Vandenberg, D. A. 2011. M67-1194, an unusually Sun-like solar twin in M67. *A&A* , **528**(Apr.), A85.
Paxton, Bill, Bildsten, Lars, Dotter, Aaron, Herwig, Falk, Lesaffre, Pierre, & Timmes, Frank. 2010 (Oct.). *MESA: Modules for Experiments in Stellar Astrophysics*. Astrophysics Source Code Library, record ascl:1010.083.
Paxton, Bill, Bildsten, Lars, Dotter, Aaron, Herwig, Falk, Lesaffre, Pierre, & Timmes, Frank. 2011. Modules for Experiments in Stellar Astrophysics (MESA). *ApJS* , **192**(1), 3.
Pelupessy, F. I., & Portegies Zwart, S. 2012. The evolution of embedded star clusters. *MNRAS* , **420**(2), 1503–1517.
Pelupessy, F. I., van der Werf, P. P., & Icke, V. 2004. Periodic bursts of star formation in irregular galaxies. *A&A* , **422**(July), 55–64.
Pelupessy, Federico I., Jänes, Jürgen, & Portegies Zwart, Simon. 2012. N-body integrators with individual time steps from Hierarchical splitting. *New Astron.* , **17**(8), 711–719.
Pichardo, Bárbara, Moreno, Edmundo, Allen, Christine, Bedin, Luigi R., Bellini, Andrea, & Pasquini, Luca. 2012. The Sun was Not Born in M67. *AJ* , **143**(3), 73.
Plummer, H. C. 1911. On the problem of distribution in globular star clusters. *MNRAS* , **71**(Mar.), 460–470.
Portegies Zwart, Simon F. 2009. The Lost Siblings of the Sun. *ApJL* , **696**(1), L13–L16.
Pourbaix, D., Nidever, D., McCarthy, C., Butler, R. P., Tinney, C. G., Marcy, G. W., Jones, H. R. A., Penny, A. J., Carter, B. D., Bouchy, F., Pepe, F., Hearnshaw, J. B., Skuljan, J., Ramm, D., & Kent, D. 2002. Constraining the difference in convective blueshift between the components of alpha Centauri with precise radial velocities. *A&A* , **386**(Apr.), 280–285.
Rein, Hanno, & Spiegel, David S. 2015. IAS15: a fast, adaptive, high-order integrator for gravitational dynamics, accurate to machine precision over a billion orbits. *MNRAS* , **446**(2), 1424–1437.
Reipurth, Bo, & Mikkola, Seppo. 2012. Formation of the widest binary stars from dynamical unfolding of triple systems. *Nat* , **492**(7428), 221–224.

Romero-Gómez, M., Athanassoula, E., Masdemont, J. J., & García-Gómez, C. 2007. The formation of spiral arms and rings in barred galaxies. *A&A* , **472**(1), 63–75.

Romero-Gómez, M., Athanassoula, E., Antoja, T., & Figueras, F. 2011. Modelling the inner disc of the Milky Way with manifolds - I. A first step. *MNRAS* , **418**(2), 1176–1193.

Salpeter, Edwin E. 1955. The Luminosity Function and Stellar Evolution. *ApJ* , **121**(Jan.), 161.

Saz Ulibarrena, Veronica, & Portegies Zwart, Simon. 2025. Reinforcement learning for adaptive time-stepping in the chaotic gravitational three-body problem. *Communications in Nonlinear Science and Numerical Simulations*, **145**(June), 108723.

Schönrich, Ralph, Binney, James, & Dehnen, Walter. 2010. Local kinematics and the local standard of rest. *MNRAS* , **403**(4), 1829–1833.

Springel, Volker. 2005. The cosmological simulation code GADGET-2. *MNRAS* , **364**(4), 1105–1134.

Stroustrup, B. 2013. *The C++ Programming Language*. 4th edn. Addison-Wesley Professional.

Taam, Ronald E., & Sandquist, Eric L. 2000. Common Envelope Evolution of Massive Binary Stars. *ARA&A* , **38**(Jan.), 113–141.

Titarchuk, Lev, & Seifina, Elena. 2016. ESO 243-49 HLX-1: scaling of X-ray spectral properties and black hole mass determination. *A&A* , **595**(Nov.), A101.

Tremaine, Scott. 2015. The Statistical Mechanics of Planet Orbits. *The Astrophysical Journal*, **807**(2), 157.

Tricarico, Pasquale. 2012 (Apr.). *ORSA: Orbit Reconstruction, Simulation and Analysis*. Astrophysics Source Code Library, record ascl:1204.013.

Verlet, Loup. 1967. Computer "Experiments" on Classical Fluids. I. Thermodynamical Properties of Lennard-Jones Molecules. *Phys. Rev.*, **159**(Jul), 98–103.

Webb, N. A., Guérou, A., Ciambur, B., Detoeuf, A., Coriat, M., Godet, O., Barret, D., Combes, F., Contini, T., Graham, Alister W., Maccarone, T. J., Mrkalj, M., Servillat, M., Schroetter, I., & Wiersema, K. 2017. Understanding the environment around the intermediate mass black hole candidate ESO 243-49 HLX-1. *A&A* , **602**(June), A103.

Webb, Natalie, Cseh, David, Lenc, Emil, Godet, Olivier, Barret, Didier, Corbel, Stephane, Farrell, Sean, Fender, Robert, Gehrels, Neil, & Heywood, Ian. 2012. Radio Detections During Two State Transitions of the Intermediate-Mass Black Hole HLX-1. *Science*, **337**(6094), 554.

Webbink, R. F. 1984. Double white dwarfs as progenitors of R Coronae Borealis stars and type I supernovae. *ApJ* , **277**(Feb.), 355–360.

Whipple, F. L. 1950. A comet model. I. The acceleration of Comet Encke. *ApJ* , **111**(Mar.), 375–394.

Winn, Joshua N., & Fabrycky, Daniel C. 2015. The Occurrence and Architecture of Exoplanetary Systems. *ARA&A* , **53**(Aug.), 409–447.

Xiang, Maosheng, Liu, Xiaowei, Shi, Jianrong, Yuan, Haibo, Huang, Yang, Chen, Bingqiu, Wang, Chun, Tian, Zhijia, Wu, Yaqian, Yang, Yong, Zhang, Huawei, Huo, Zhiying, & Ren, Juanjuan. 2017. The Ages and Masses of a Million Galactic-disk Main-sequence Turnoff and Subgiant Stars from the LAMOST Galactic Spectro-

scopic Surveys. *ApJS* , **232**(1), 2.

Yoshida, Haruo. 1990. Construction of higher order symplectic integrators. *Physics Letters A*, **150**(5-7), 262–268.

Zemp, Marcel. 2014 (July). *Halogen: Multimass spherical structure models for N-body simulations*. Astrophysics Source Code Library, record ascl:1407.020.

Zemp, Marcel, Moore, Ben, Stadel, Joachim, Carollo, C. Marcella, & Madau, Piero. 2008. Multimass spherical structure models for N-body simulations. *MNRAS* , **386**(3), 1543–1556.